VNRVJIET

NAME OF LAB: _____ WEEK: _____

EXPERIMENT NAME:

WEEK - 1

Title of the Experiment: Self-Driving Car using Deep Learning

Objective of the experiment:

To develop a self-driving car model using deep learning techniques by leveraging a convolutional neural network (CNN) for steering angle prediction. Train and evaluate the model on real work self-driving datasets.

Implementation Steps and Code:

a. Problem Definition and Dataset

- Understand the problem of autonomous driving.
- Use the Udacity self-driving car dataset (or any open-source dataset)

b. Data understanding, analysis of files and folders

```
import os, random, ntpath, subprocess
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
import cv2

from sklearn.utils import shuffle
from sklearn.model_selection import train_test_split

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Conv2D, Dense, MaxPooling2D, Dropout, Flatten
from tensorflow.keras.optimizers import Adam

random.seed(0); np.random.seed(0); tf.random.set_seed(0)
```

```
datadir = "data/selfdrive"
if not os.path.exists(os.path.join(datadir, "driving_log.csv")):
    subprocess.run(["git", "clone", "--depth", "1", "https://github.com/Aman-py/data.git",
datadir], check=True)
print("Dataset folder ready:", os.path.exists(datadir))
print("Images in IMG/:", len(os.listdir(os.path.join(datadir, "IMG"))))
```

```
Dataset folder ready: True
Images in IMG/: 21777
```

```
columns = ["center", "left", "right", "steering", "throttle", "reverse", "speed"]
dataset = pd.read_csv(os.path.join(datadir, "driving_log.csv"), names=columns)
print("Number of samples:", len(dataset))
```

VNRVJIET

NAME OF LAB: _____ WEEK: _____

EXPERIMENT NAME:

```
print("Columns:", list(dataset.columns))
dataset.head()
```

```
Number of samples: 7259
Columns: ['center', 'left', 'right', 'steering', 'throttle', 'reverse', 'speed']
```

```
                                              center  \
0  /home/aman-py/Desktop/Self-Driving-Car/data/IM...
1  /home/aman-py/Desktop/Self-Driving-Car/data/IM...
2  /home/aman-py/Desktop/Self-Driving-Car/data/IM...
3  /home/aman-py/Desktop/Self-Driving-Car/data/IM...
4  /home/aman-py/Desktop/Self-Driving-Car/data/IM...

                                               left  \
0   /home/aman-py/Desktop/Self-Driving-Car/data/I...
1   /home/aman-py/Desktop/Self-Driving-Car/data/I...
2   /home/aman-py/Desktop/Self-Driving-Car/data/I...
3   /home/aman-py/Desktop/Self-Driving-Car/data/I...
4   /home/aman-py/Desktop/Self-Driving-Car/data/I...

                                              right  steering  throttle  \
0   /home/aman-py/Desktop/Self-Driving-Car/data/I...       0.0       0.0
1   /home/aman-py/Desktop/Self-Driving-Car/data/I...       0.0       0.0
2   /home/aman-py/Desktop/Self-Driving-Car/data/I...       0.0       0.0
3   /home/aman-py/Desktop/Self-Driving-Car/data/I...       0.0       0.0
4   /home/aman-py/Desktop/Self-Driving-Car/data/I...       0.0       0.0

   reverse     speed
0      0.0  0.000078
1      0.0  0.000078
2      0.0  0.000079
3      0.0  0.000078
4      0.0  0.000079
```

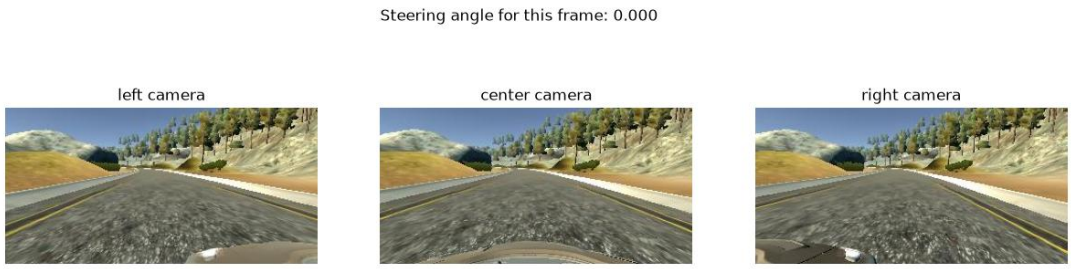
```
print(dataset[["steering", "throttle", "speed"]].describe().round(4))
print("\nShare of frames with |steering| < 0.04:", round((dataset["steering"].abs() <
0.04).mean() * 100, 1), "%")
```

```
        steering   throttle      speed
count  7259.0000  7259.0000  7259.0000
mean     -0.0177     0.2831    19.9303
std       0.1832     0.3994     7.6456
min      -1.0000     0.0000     0.0001
25%       0.0000     0.0000    14.0720
50%       0.0000     0.0000    19.5122
75%       0.0000     0.5466    27.1582
max       1.0000     1.0000    30.6142

Share of frames with |steering| < 0.04: 75.1 %
```

```
row = dataset.iloc[500]
fig, axs = plt.subplots(1, 3, figsize=(15, 4))
for ax, cam in zip(axs, ["left", "center", "right"]):
    img = mpimg.imread(os.path.join(datadir, "IMG", ntpath.split(row[cam])[1]))
    ax.imshow(img); ax.set_title(f"{cam} camera"); ax.axis("off")
plt.suptitle(f"Steering angle for this frame: {row['steering']:.3f}")
```

```
plt.show()
print("Image shape (height, width, channels):", img.shape)
```




```
Image shape (height, width, channels): (160, 320, 3)
```

VNRVJIET

NAME OF LAB: _____ WEEK: _____

EXPERIMENT NAME:

WEEK - 2

c. Dashcam images and steering analysis

```
def removePath(path):
    base, tail = ntpath.split(path)
    return tail

dataset["center"] = dataset["center"].apply(removePath)
dataset["left"]   = dataset["left"].apply(removePath)
dataset["right"]  = dataset["right"].apply(removePath)
dataset.head()
```

```
                            center                                left  \
0  center_2018_10_17_16_36_30_865.jpg  left_2018_10_17_16_36_30_865.jpg
1  center_2018_10_17_16_36_30_966.jpg  left_2018_10_17_16_36_30_966.jpg
2  center_2018_10_17_16_36_31_098.jpg  left_2018_10_17_16_36_31_098.jpg
3  center_2018_10_17_16_36_31_253.jpg  left_2018_10_17_16_36_31_253.jpg
4  center_2018_10_17_16_36_31_366.jpg  left_2018_10_17_16_36_31_366.jpg

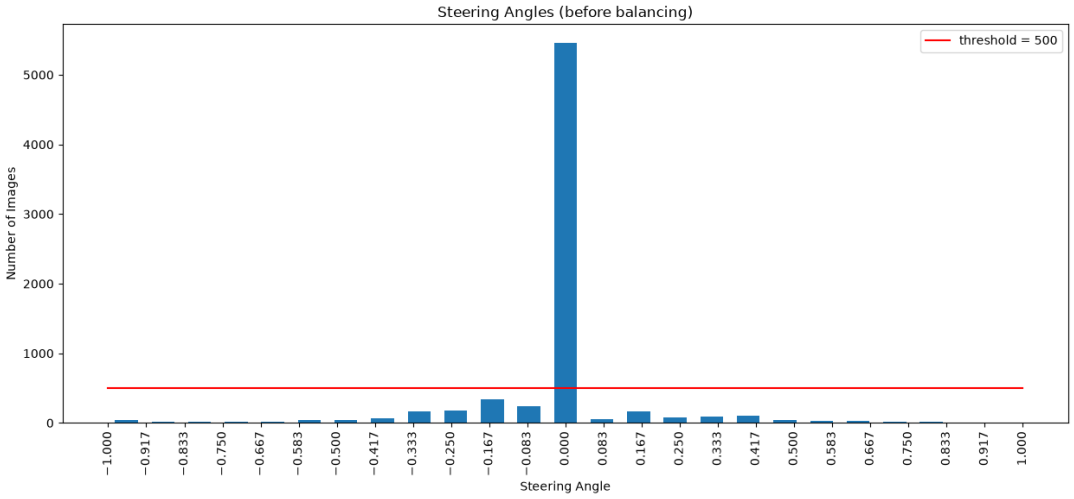
                             right  steering  throttle  reverse     speed
0  right_2018_10_17_16_36_30_865.jpg       0.0       0.0      0.0  0.000078
1  right_2018_10_17_16_36_30_966.jpg       0.0       0.0      0.0  0.000078
2  right_2018_10_17_16_36_31_098.jpg       0.0       0.0      0.0  0.000079
3  right_2018_10_17_16_36_31_253.jpg       0.0       0.0      0.0  0.000078
4  right_2018_10_17_16_36_31_366.jpg       0.0       0.0      0.0  0.000079
```

```
num_bins = 25
hist, bins = np.histogram(dataset["steering"], num_bins)
print("Histogram counts:", hist)
print("Bin edges:", bins.round(2))

center = (bins[:-1] + bins[1:]) * 0.5       # midpoint of each bin, used as bar positions
print("Bin centers:", center.round(2))
```

```
Histogram counts: [  48   16   13   15   21   36   48   69  161  174  337  239 5454   56
  160   82   97   99   37   25   32   15   12    5    8]
Bin edges: [-1.   -0.92 -0.84 -0.76 -0.68 -0.6  -0.52 -0.44 -0.36 -0.28 -0.2  -0.12
 -0.04  0.04  0.12  0.2   0.28  0.36  0.44  0.52  0.6   0.68  0.76  0.84
  0.92  1.  ]
Bin centers: [-0.96 -0.88 -0.8  -0.72 -0.64 -0.56 -0.48 -0.4  -0.32 -0.24 -0.16 -0.08
  0.    0.08  0.16  0.24  0.32  0.4   0.48  0.56  0.64  0.72  0.8   0.88
  0.96]
```

```
threshold = 500
plt.figure(figsize=(15, 6))
plt.bar(center, hist, width=0.05)
plt.xticks(np.linspace(-1, 1, 25), rotation=90)
plt.title("Steering Angles (before balancing)")
plt.xlabel("Steering Angle"); plt.ylabel("Number of Images")
plt.plot((dataset["steering"].min(), dataset["steering"].max()), (threshold, threshold),
color="red", label="threshold = 500")
plt.legend(); plt.show()
```



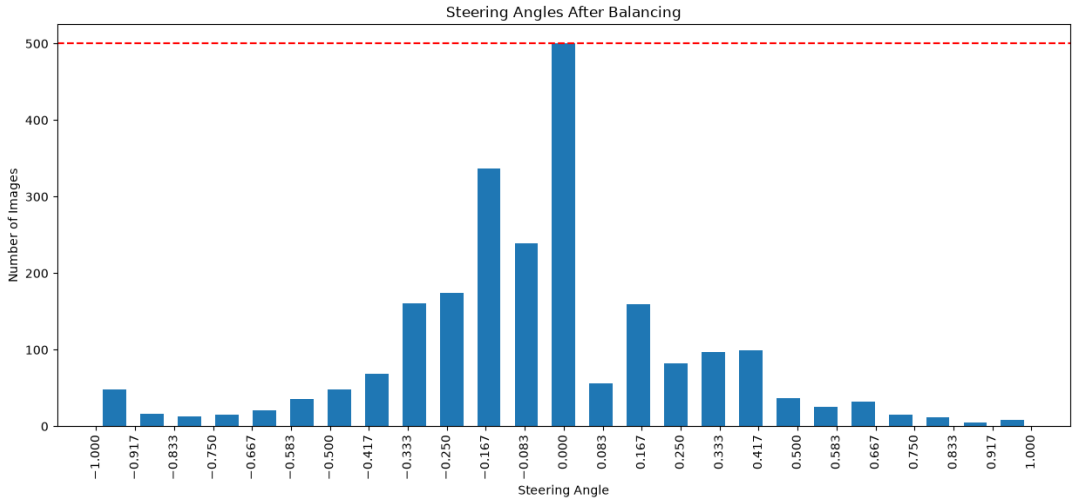

```
bin_id = np.clip(np.digitize(dataset["steering"], bins[1:-1]), 0, num_bins - 1)

remove_list = []
for i in range(num_bins):
    bin_indices = np.where(bin_id == i)[0]                # rows that fall in bin i
    bin_indices = shuffle(bin_indices, random_state=42)   # random order
    remove_list.extend(bin_indices[threshold:])           # everything beyond the first 500 is
surplus

print("Samples to remove:", len(remove_list))
dataset = dataset.drop(dataset.index[remove_list]).reset_index(drop=True)
print("Dataset size after balancing:", len(dataset))
```

```
Samples to remove: 4954
Dataset size after balancing: 2305
```

```
hist, _ = np.histogram(dataset["steering"], num_bins)
plt.figure(figsize=(15, 6))
plt.bar(center, hist, width=0.05)
plt.xticks(np.linspace(-1, 1, 25), rotation=90)
plt.title("Steering Angles After Balancing")
plt.xlabel("Steering Angle"); plt.ylabel("Number of Images")
plt.axhline(threshold, linestyle="--", color="red"); plt.show()
dataset.iloc[1]
```




```
center      center_2018_10_17_16_36_32_664.jpg
left          left_2018_10_17_16_36_32_664.jpg
right        right_2018_10_17_16_36_32_664.jpg
steering                                   0.0
throttle                                   0.0
reverse                                    0.0
speed                                 0.000078
Name: 1, dtype: object
```

```
def loadImageSteering(datadir, dataset):
    imagePath, steeringPath = [], []
    for i in range(len(dataset)):
        center_image = dataset.iloc[i]["center"]
        steering = float(dataset.iloc[i]["steering"])
        imagePath.append(os.path.join(datadir, center_image))
        steeringPath.append(steering)
    return np.asarray(imagePath), np.asarray(steeringPath)

imagePath, steeringPath = loadImageSteering(os.path.join(datadir, "IMG"), dataset)
print("First image:", imagePath[0])
print("Number of images:", len(imagePath), "| steering values:", len(steeringPath))
print("File exists:", os.path.exists(imagePath[0]))
```

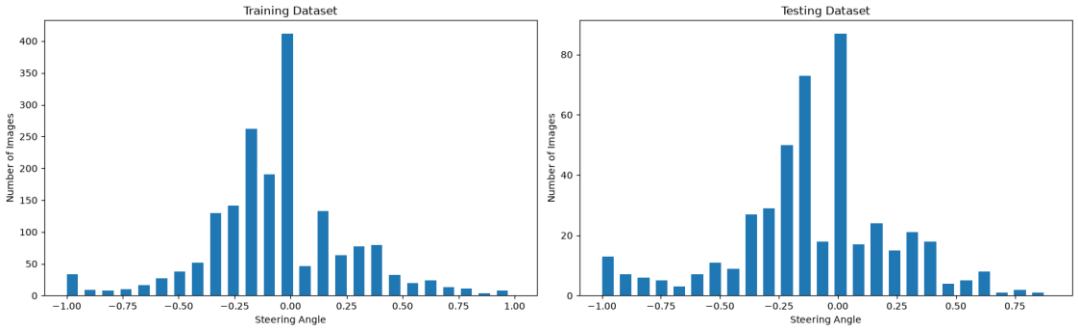
```
First image: data/selfdrive\IMG\center_2018_10_17_16_36_31_862.jpg
Number of images: 2305 | steering values: 2305
File exists: True
```

d. EDA steering angles

```
x_train, x_test, y_train, y_test = train_test_split(imagePath, steeringPath, test_size=0.2,
random_state=6)
print("Training samples:", len(x_train))
print("Testing samples:", len(x_test))
```

```
Training samples: 1844
Testing samples: 461
```

```
fig, axs = plt.subplots(1, 2, figsize=(16, 5), sharey=False)
for ax, y, name in zip(axs, [y_train, y_test], ["Training Dataset", "Testing Dataset"]):
    ax.hist(y, bins=num_bins, width=0.05)
    ax.set_title(name); ax.set_xlabel("Steering Angle"); ax.set_ylabel("Number of Images")
plt.tight_layout(); plt.show()
```



e. Mean baseline model

```
mean_steering = np.mean(y_train)
baseline_predictions = np.full(len(y_test), mean_steering)
baseline_mse = np.mean((y_test - baseline_predictions) ** 2)

print("Mean steering angle (train):", mean_steering)
print("Baseline MSE on test set:", baseline_mse)
```

```
Mean steering angle (train): -0.04546927379229935
Baseline MSE on test set: 0.11740117013233528
```

VNRVJIET

NAME OF LAB: _____ WEEK: _____

EXPERIMENT NAME:

WEEK - 3

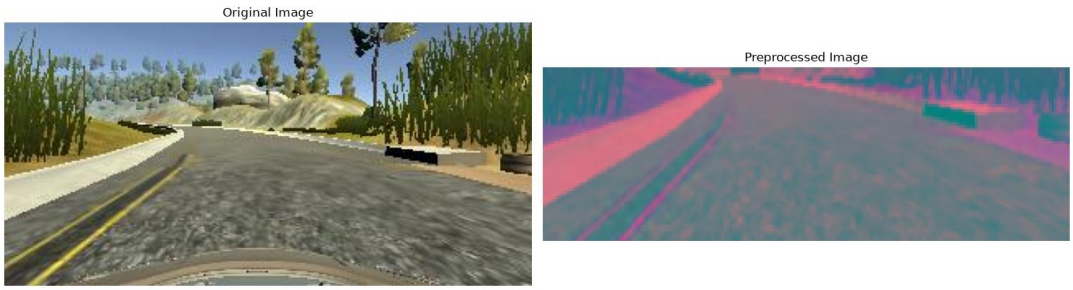
f. Deep learning model, deep learning for regression

```
def imagePreprocessing(img_path):
    img = mpimg.imread(img_path)                    # read image (RGB)
    img = img[60:135, :, :]                         # crop away sky and car bonnet
    img = cv2.cvtColor(img, cv2.COLOR_RGB2YUV)      # RGB -> YUV (as in NVIDIA's paper)
    img = cv2.GaussianBlur(img, (3, 3), 0)          # light denoising
    img = cv2.resize(img, (200, 66))                # NVIDIA input size (width, height)
    img = img / 255.0                               # scale pixels to [0, 1]
    return img
```

```
sample_image = x_train[0]
original = mpimg.imread(sample_image)
processed = imagePreprocessing(sample_image)

fig, axs = plt.subplots(1, 2, figsize=(15, 6))
axs[0].imshow(original);  axs[0].set_title("Original Image");     axs[0].axis("off")
axs[1].imshow(processed); axs[1].set_title("Preprocessed Image"); axs[1].axis("off")
plt.tight_layout(); plt.show()

print("Original shape:", original.shape)
print("Processed shape:", processed.shape, "| value range:", processed.min().round(2), "-",
processed.max().round(2))
print("Steering angle of this image:", y_train[0])
```




```
Original shape: (160, 320, 3)
Processed shape: (66, 200, 3) | value range: 0.02 - 0.92
Steering angle of this image: -0.1499941
```

```
def batch_generator(image_paths, steering_values, batch_size=32):
    while True:                                     # Keras keeps pulling batches, so loop
forever
        indices = np.random.permutation(len(image_paths))      # reshuffle every pass
        for start in range(0, len(image_paths), batch_size):
            batch_indices = indices[start:start + batch_size]
            images, angles = [], []
            for index in batch_indices:
                images.append(imagePreprocessing(image_paths[index]))
                angles.append(steering_values[index])
            yield np.array(images, dtype=np.float32), np.array(angles, dtype=np.float32)
```

```
batch_size = 32
train_generator = batch_generator(x_train, y_train, batch_size)
validation_generator = batch_generator(x_test, y_test, batch_size)

X_batch, Y_batch = next(train_generator)
print("Batch image shape:", X_batch.shape, "  -> (batch, height, width, channels)")
print("Batch steering shape:", Y_batch.shape)
```

```
Batch image shape: (32, 66, 200, 3)   -> (batch, height, width, channels)
Batch steering shape: (32,)
```

h. NVIDIA's end-to-end CNN model

```
def nvidiaModel():
    model = Sequential([
        Input(shape=(66, 200, 3)),
        Conv2D(24, (5, 5), strides=(2, 2), activation="elu"),
        Conv2D(36, (5, 5), strides=(2, 2), activation="elu"),
        Conv2D(48, (5, 5), strides=(2, 2), activation="elu"),
        Conv2D(64, (3, 3), activation="elu"),
        Conv2D(64, (3, 3), activation="elu"),
        Dropout(0.5),
        Flatten(),
        Dense(100, activation="elu"), Dropout(0.5),
        Dense(50, activation="elu"),  Dropout(0.5),
        Dense(10, activation="elu"),  Dropout(0.5),
        Dense(1),                                    # regression output: steering angle
    ])
    model.compile(optimizer=Adam(learning_rate=1e-3), loss="mse")
    return model

model = nvidiaModel()
model.summary()
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.
```

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 31, 98, 24)	1,824
conv2d_1 (Conv2D)	(None, 14, 47, 36)	21,636
conv2d_2 (Conv2D)	(None, 5, 22, 48)	43,248
conv2d_3 (Conv2D)	(None, 3, 20, 64)	27,712
conv2d_4 (Conv2D)	(None, 1, 18, 64)	36,928
dropout (Dropout)	(None, 1, 18, 64)	0

flatten (Flatten)	(None, 1152)	0
dense (Dense)	(None, 100)	115,300
dropout_1 (Dropout)	(None, 100)	0
dense_1 (Dense)	(None, 50)	5,050
dropout_2 (Dropout)	(None, 50)	0
dense_2 (Dense)	(None, 10)	510
dropout_3 (Dropout)	(None, 10)	0
dense_3 (Dense)	(None, 1)	11

Total params: 252,219 (985.23 KB)

Trainable params: 252,219 (985.23 KB)

Non-trainable params: 0 (0.00 B)

```
pred = model.predict(X_batch, verbose=0)
print("Input batch:", X_batch.shape, "-> output:", pred.shape)
print("Untrained predictions (first 5):", pred.flatten()[:5].round(4))
print("Total parameters:", f"{model.count_params():,}")
```

```
Input batch: (32, 66, 200, 3) -> output: (32, 1)
Untrained predictions (first 5): [0.9836 1.0027 0.9548 0.9868 1.0255]
Total parameters: 252,219
```

WEEK - 4

i. Train the model

```
steps_per_epoch = len(x_train) // batch_size
validation_steps = len(x_test) // batch_size
print("Steps per epoch:", steps_per_epoch)
print("Validation steps:", validation_steps)
```

```
Steps per epoch: 57
Validation steps: 14
```

```
history = model.fit(
    train_generator,
    steps_per_epoch=steps_per_epoch,
    validation_data=validation_generator,
    validation_steps=validation_steps,
    epochs=30,
    verbose=2,
)
```

```
Epoch 1/30
57/57 - 9s - 158ms/step - loss: 0.4086 - val_loss: 0.1133
Epoch 2/30
57/57 - 3s - 60ms/step - loss: 0.1240 - val_loss: 0.0953
Epoch 3/30
57/57 - 4s - 70ms/step - loss: 0.1140 - val_loss: 0.0832
Epoch 4/30
57/57 - 4s - 66ms/step - loss: 0.1056 - val_loss: 0.0939
Epoch 5/30
57/57 - 4s - 71ms/step - loss: 0.0994 - val_loss: 0.0754
Epoch 6/30
57/57 - 4s - 77ms/step - loss: 0.0877 - val_loss: 0.0823
Epoch 7/30
57/57 - 5s - 85ms/step - loss: 0.0837 - val_loss: 0.0838
Epoch 8/30
57/57 - 4s - 76ms/step - loss: 0.0819 - val_loss: 0.0699
Epoch 9/30
57/57 - 4s - 73ms/step - loss: 0.0829 - val_loss: 0.0745
Epoch 10/30
57/57 - 4s - 71ms/step - loss: 0.0752 - val_loss: 0.0729
Epoch 11/30
57/57 - 3s - 61ms/step - loss: 0.0725 - val_loss: 0.0661
Epoch 12/30
57/57 - 4s - 67ms/step - loss: 0.0732 - val_loss: 0.0787
Epoch 13/30
57/57 - 4s - 73ms/step - loss: 0.0680 - val_loss: 0.0784
Epoch 14/30
57/57 - 5s - 87ms/step - loss: 0.0724 - val_loss: 0.0701
Epoch 15/30
57/57 - 4s - 76ms/step - loss: 0.0674 - val_loss: 0.0711
Epoch 16/30
57/57 - 4s - 72ms/step - loss: 0.0639 - val_loss: 0.0800
Epoch 17/30
57/57 - 4s - 74ms/step - loss: 0.0624 - val_loss: 0.0870
```

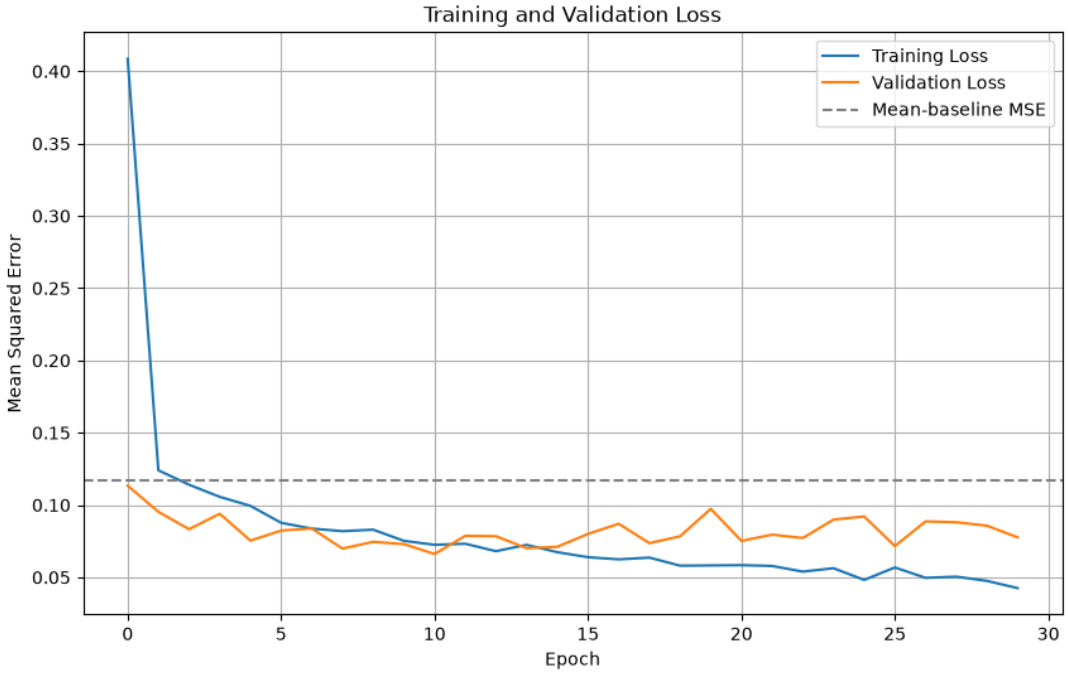
```
Epoch 18/30
57/57 - 5s - 87ms/step - loss: 0.0636 - val_loss: 0.0736
Epoch 19/30
57/57 - 5s - 90ms/step - loss: 0.0580 - val_loss: 0.0784
Epoch 20/30
57/57 - 5s - 92ms/step - loss: 0.0582 - val_loss: 0.0973
Epoch 21/30
57/57 - 6s - 101ms/step - loss: 0.0584 - val_loss: 0.0752
Epoch 22/30
57/57 - 6s - 98ms/step - loss: 0.0578 - val_loss: 0.0795
Epoch 23/30
57/57 - 5s - 93ms/step - loss: 0.0539 - val_loss: 0.0772
Epoch 24/30
57/57 - 6s - 107ms/step - loss: 0.0563 - val_loss: 0.0899
Epoch 25/30
57/57 - 5s - 93ms/step - loss: 0.0482 - val_loss: 0.0920
Epoch 26/30
57/57 - 5s - 92ms/step - loss: 0.0568 - val_loss: 0.0716
Epoch 27/30
57/57 - 5s - 90ms/step - loss: 0.0496 - val_loss: 0.0886
Epoch 28/30
57/57 - 5s - 84ms/step - loss: 0.0504 - val_loss: 0.0881
Epoch 29/30
57/57 - 5s - 87ms/step - loss: 0.0475 - val_loss: 0.0857
Epoch 30/30
57/57 - 6s - 103ms/step - loss: 0.0425 - val_loss: 0.0776
```

```
plt.figure(figsize=(10, 6))
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.axhline(baseline_mse, color="gray", linestyle="--", label="Mean-baseline MSE")
plt.title("Training and Validation Loss"); plt.xlabel("Epoch"); plt.ylabel("Mean Squared
Error")
plt.legend(); plt.grid(True); plt.show()
print("Final training loss:", round(history.history["loss"][-1], 5))
print("Final validation loss:", round(history.history["val_loss"][-1], 5))
```

VNRVJIET

NAME OF LAB: _____ **WEEK:** _____

EXPERIMENT NAME:




```
Final training loss: 0.0425
Final validation loss: 0.07764
```

j. Test and visualize the output and its extensions

```
x_test_processed = np.array([imagePreprocessing(p) for p in x_test], dtype=np.float32)
print("Test data shape:", x_test_processed.shape)

test_loss = model.evaluate(x_test_processed, y_test, verbose=0)
print("Test MSE:", test_loss)
```

```
Test data shape: (461, 66, 200, 3)
Test MSE: 0.08612313866615295
```

```
predictions = model.predict(x_test_processed, verbose=0).flatten()

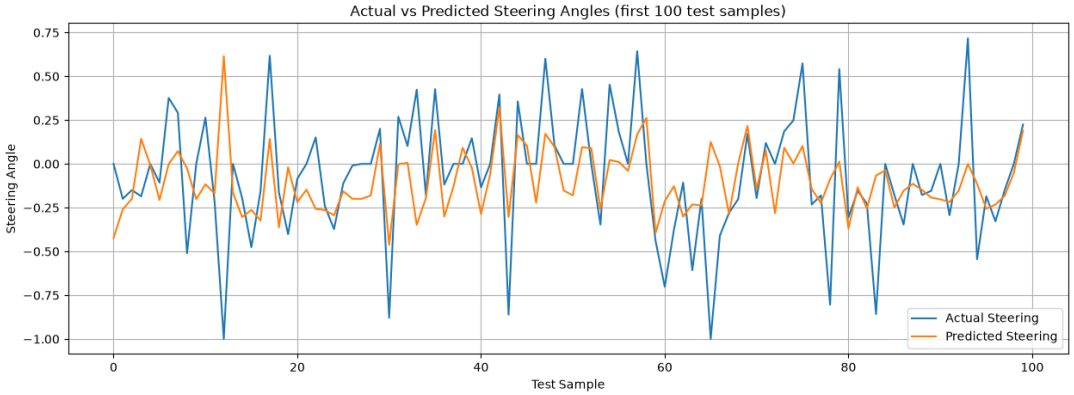
comparison = pd.DataFrame({"Actual Steering": y_test, "Predicted Steering": predictions})
comparison["Abs Error"] = (comparison["Actual Steering"] - comparison["Predicted
Steering"]).abs()
print("Mean absolute error:", comparison["Abs Error"].mean().round(4))
comparison.head(10)
```

```
Mean absolute error: 0.2064
```

	Actual Steering	Predicted Steering	Abs Error
0	0.000000	-0.423103	0.423103
1	-0.199455	-0.260063	0.060608
2	-0.149508	-0.198972	0.049463
3	-0.185046	0.143347	0.328393
4	0.000000	-0.006996	0.006996
5	-0.106884	-0.206558	0.099674
6	0.376845	-0.001468	0.378313

```
7         0.291775           0.072853   0.218922
8        -0.510579          -0.025407   0.485172
9         0.000000          -0.201758   0.201758
```

```
plt.figure(figsize=(15, 5))
plt.plot(y_test[:100], label="Actual Steering")
plt.plot(predictions[:100], label="Predicted Steering")
plt.title("Actual vs Predicted Steering Angles (first 100 test samples)")
plt.xlabel("Test Sample"); plt.ylabel("Steering Angle"); plt.legend(); plt.grid(True);
plt.show()
```

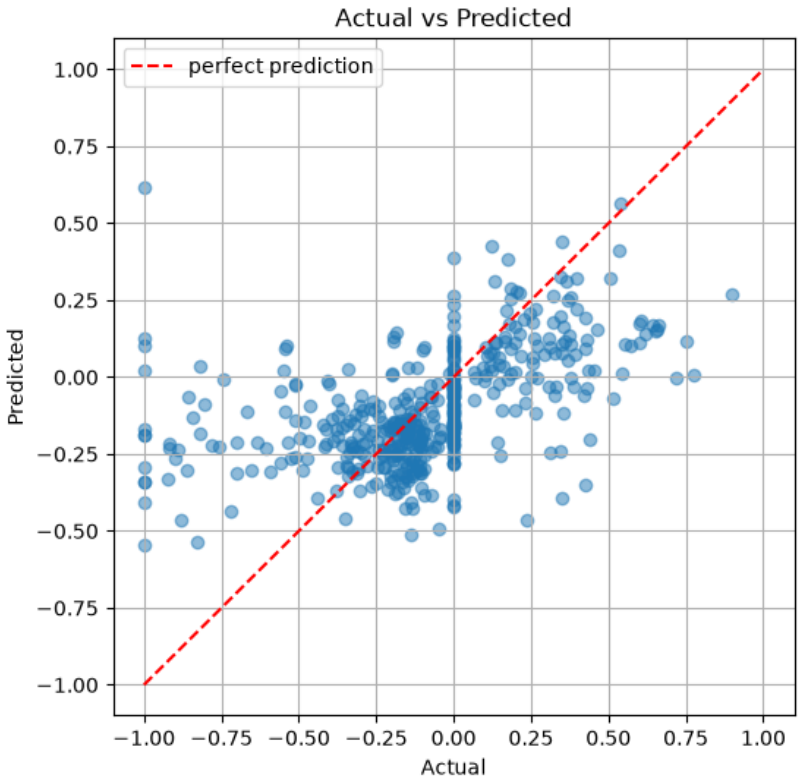



```
plt.figure(figsize=(6, 6))
plt.scatter(y_test, predictions, alpha=0.5)
plt.plot([-1, 1], [-1, 1], "r--", label="perfect prediction")
plt.xlabel("Actual"); plt.ylabel("Predicted"); plt.title("Actual vs Predicted"); plt.legend();
plt.grid(True); plt.show()
print("Correlation actual vs predicted:", np.corrcoef(y_test, predictions)[0, 1].round(3))
```

VNRVJIET

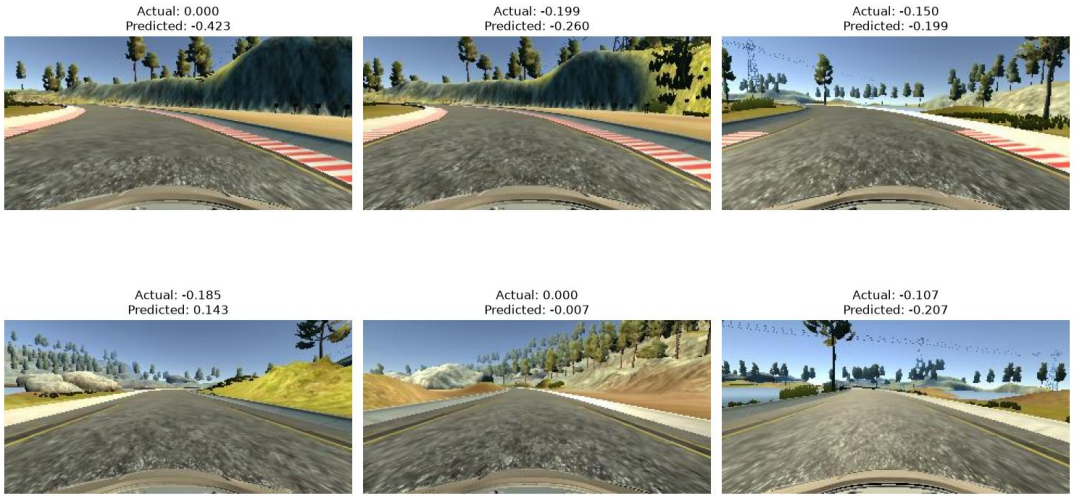
NAME OF LAB: _____ **WEEK:** _____

EXPERIMENT NAME:




```
Correlation actual vs predicted: 0.503
```

```
plt.figure(figsize=(15, 9))
for i in range(6):
    plt.subplot(2, 3, i + 1)
    plt.imshow(mpimg.imread(x_test[i])); plt.axis("off")
    plt.title(f"Actual: {y_test[i]:.3f}\nPredicted: {predictions[i]:.3f}")
plt.tight_layout(); plt.show()
```




```
print("Mean Baseline MSE :", baseline_mse)
```

```
print("NVIDIA CNN Test MSE:", test_loss)
print("Improvement over baseline:", round((1 - test_loss / baseline_mse) * 100, 1), "%")
```

```
Mean Baseline MSE : 0.11740117013233528
NVIDIA CNN Test MSE: 0.08612313866615295
Improvement over baseline: 26.6 %
```

```
model.save("car.keras")
print("Model saved successfully.")

reloaded = tf.keras.models.load_model("car.keras")
print("Reloaded predictions identical:", np.allclose(reloaded.predict(x_test_processed[:10],
verbose=0).flatten(), predictions[:10], atol=1e-5))
```

```
Model saved successfully.
Reloaded predictions identical: True
```

VNRVJIET

NAME OF LAB: _____ WEEK: _____

EXPERIMENT NAME:

WEEK - 5

Title of the Experiment: Human Activity Recognition (HAR) using Deep Learning

Objective of the experiment:

To implement Human Activity Recognition (HAR) using classical Machine Learning and Deep Learning techniques, utilizing a public dataset such as the UCI HAR dataset.

a. Problem definition

Human Activity Recognition (HAR) is a classification problem where data collected from motion sensors (such as accelerometers and gyroscopes) is used to recognize human activities like walking, sitting, standing, etc. HAR has applications in health monitoring, sports analysis, and smart home automation.

Data collected from 30 volunteers performing 6 activities:

- Walking
- Walking Upstairs
- Walking Downstairs
- Sitting
- Standing
- Laying

Features extracted from time-domain & frequency-domain signals.

```
import os
BASE = "data/har_raw/UCI HAR Dataset/"
for root, dirs, files in os.walk(BASE):
    if "Inertial Signals" in root:
        print(root.replace(BASE, ""), "->", len(files), "raw signal files (e.g.", files[0],
")")
        continue
    print(root.replace(BASE, "") or ".", "->", files)
```

```
. -> ['.DS_Store', 'activity_labels.txt', 'features.txt', 'features_info.txt', 'README.txt']
test -> ['subject_test.txt', 'X_test.txt', 'y_test.txt']
test\Inertial Signals -> 9 raw signal files (e.g. body_acc_x_test.txt )
train -> ['subject_train.txt', 'X_train.txt', 'y_train.txt']
train\Inertial Signals -> 9 raw signal files (e.g. body_acc_x_train.txt )
```

```
import pandas as pd
activity_labels = pd.read_csv(BASE + "activity_labels.txt", sep=r"\s+", header=None,
names=["id", "activity"])
```

```
features = pd.read_csv(BASE + "features.txt", sep=r"\s+", header=None, names=["idx",
"feature"])
print(activity_labels.to_string(index=False))
print("\nTotal features:", len(features))
print("First 8 feature names:")
print(features["feature"].head(8).tolist())

subj_train = pd.read_csv(BASE + "train/subject_train.txt", header=None)[0]
subj_test = pd.read_csv(BASE + "test/subject_test.txt", header=None)[0]
print("\nVolunteers in train:", subj_train.nunique(), "| in test:", subj_test.nunique(), "|
overlap:", len(set(subj_train) & set(subj_test)))
```

```
 id           activity
  1            WALKING
  2   WALKING_UPSTAIRS
  3 WALKING_DOWNSTAIRS
  4            SITTING
  5           STANDING
  6             LAYING

Total features: 561
First 8 feature names:
['tBodyAcc-mean()-X', 'tBodyAcc-mean()-Y', 'tBodyAcc-mean()-Z', 'tBodyAcc-std()-X', 'tBodyAcc-
std()-Y', 'tBodyAcc-std()-Z', 'tBodyAcc-mad()-X', 'tBodyAcc-mad()-Y']

Volunteers in train: 21 | in test: 9 | overlap: 0
```

```
names = features["feature"]
print("Time-domain features     (start with 't'):", names.str.startswith("t").sum())
print("Frequency-domain features (start with 'f'):", names.str.startswith("f").sum())
print("Other (angle features)   :", (~names.str[0].isin(["t", "f"])).sum())
```

```
Time-domain features     (start with 't'): 265
Frequency-domain features (start with 'f'): 289
Other (angle features)   : 7
```

VNRVJIET

NAME OF LAB: _____ WEEK: _____

EXPERIMENT NAME:

WEEK - 6

b. Dataset understanding

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import (accuracy_score, classification_report, confusion_matrix,
                             precision_score, recall_score, f1_score)
```

```
# Local paths (the record used /content/... on Google Colab)
BASE = "data/har_raw/UCI HAR Dataset/"
X_train_path = BASE + "train/X_train.txt"
y_train_path = BASE + "train/y_train.txt"
X_test_path = BASE + "test/X_test.txt"
y_test_path = BASE + "test/y_test.txt"
features_path = BASE + "features.txt"
activities_path = BASE + "activity_labels.txt"

# Feature names (2nd column of features.txt)
features = pd.read_csv(features_path, sep=r"\s+", header=None)
feature_names = features.iloc[:, 1].tolist()

# Feature matrices: whitespace-separated numbers, no header
X_train = pd.read_csv(X_train_path, sep=r"\s+", header=None)
X_test = pd.read_csv(X_test_path, sep=r"\s+", header=None)

# The file has a few duplicated feature names (e.g. bandsEnergy), so we make them unique by
suffixing the column index
seen, unique_names = {}, []
for i, n in enumerate(feature_names):
    unique_names.append(n if feature_names.count(n) == 1 else f"{n}_{i}")
feature_names = unique_names
X_train.columns = feature_names
X_test.columns = feature_names

# Labels
y_train = np.loadtxt(y_train_path, dtype=int)
y_test = np.loadtxt(y_test_path, dtype=int)

# Activity id -> name, keeping names that contain underscores/spaces
with open(activities_path, "r") as file:
    activity_names = {int(line.split(maxsplit=1)[0]): line.split(maxsplit=1)[1].strip() for
line in file if line.strip()}

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
print("Number of features:", len(feature_names))
print("Activity names:", activity_names)
```

```
Training data: (7352, 561)
Testing data: (2947, 561)
Number of features: 561
Activity names: {1: 'WALKING', 2: 'WALKING_UPSTAIRS', 3: 'WALKING_DOWNSTAIRS', 4: 'SITTING', 5:
'STANDING', 6: 'LAYING'}
```

```
X_train.iloc[:5, :8]
```

```
   tBodyAcc-mean()-X  tBodyAcc-mean()-Y  tBodyAcc-mean()-Z  tBodyAcc-std()-X  \
0           0.288585          -0.020294          -0.132905         -0.995279
1           0.278419          -0.016411          -0.123520         -0.998245
2           0.279653          -0.019467          -0.113462         -0.995380
3           0.279174          -0.026201          -0.123283         -0.996091
4           0.276629          -0.016570          -0.115362         -0.998139

   tBodyAcc-std()-Y  tBodyAcc-std()-Z  tBodyAcc-mad()-X  tBodyAcc-mad()-Y
0         -0.983111         -0.913526         -0.995112         -0.983185
1         -0.975300         -0.960322         -0.998807         -0.974914
2         -0.967187         -0.978944         -0.996520         -0.963668
3         -0.983403         -0.990675         -0.997099         -0.982750
4         -0.980817         -0.990482         -0.998321         -0.979672
```

```
print(X_train.iloc[:, :6].describe().round(3))
print("\nTrain label counts:", dict(zip(*np.unique(y_train, return_counts=True))))
```

```
       tBodyAcc-mean()-X  tBodyAcc-mean()-Y  tBodyAcc-mean()-Z  \
count           7352.000           7352.000           7352.000
mean               0.274             -0.018             -0.109
std                0.070              0.041              0.057
min               -1.000             -1.000             -1.000
25%                0.263             -0.025             -0.121
50%                0.277             -0.017             -0.109
75%                0.288             -0.011             -0.098
max                1.000              1.000              1.000

       tBodyAcc-std()-X  tBodyAcc-std()-Y  tBodyAcc-std()-Z
count          7352.000          7352.000          7352.000
mean             -0.605            -0.511            -0.605
std               0.449             0.503             0.419
min              -1.000            -1.000            -1.000
25%              -0.993            -0.978            -0.980
50%              -0.946            -0.852            -0.859
75%              -0.243            -0.034            -0.262
max               1.000             0.916             1.000

Train label counts: {1: 1226, 2: 1073, 3: 986, 4: 1286, 5: 1374, 6: 1407}
```

c. Data cleaning and pre-processing

```
print("Missing values in training data:", X_train.isnull().sum().sum())
print("Missing values in testing data:", X_test.isnull().sum().sum())
print("Duplicate training rows:", X_train.duplicated().sum())
print("Infinite values in training data:", np.isinf(X_train.to_numpy()).sum())
```

```
Missing values in training data: 0
```

```
Missing values in testing data: 0
Duplicate training rows: 0
Infinite values in training data: 0
```

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)      # learn mean/std on TRAIN only
X_test_scaled = scaler.transform(X_test)            # reuse the train statistics on test
y_train_encoded = y_train - 1                       # ids 1-6 -> 0-5 (what Keras / sklearn
metrics expect)
y_test_encoded = y_test - 1

print("Preprocessing completed.")
print("Training features:", X_train_scaled.shape)
print("Testing features:", X_test_scaled.shape)
print("Train mean (should be ~0):", X_train_scaled.mean().round(4), "| std (~1):",
X_train_scaled.std().round(4))
print("Encoded labels:", np.unique(y_train_encoded))
```

```
Preprocessing completed.
Training features: (7352, 561)
Testing features: (2947, 561)
Train mean (should be ~0): 0.0 | std (~1): 1.0
Encoded labels: [0 1 2 3 4 5]
```

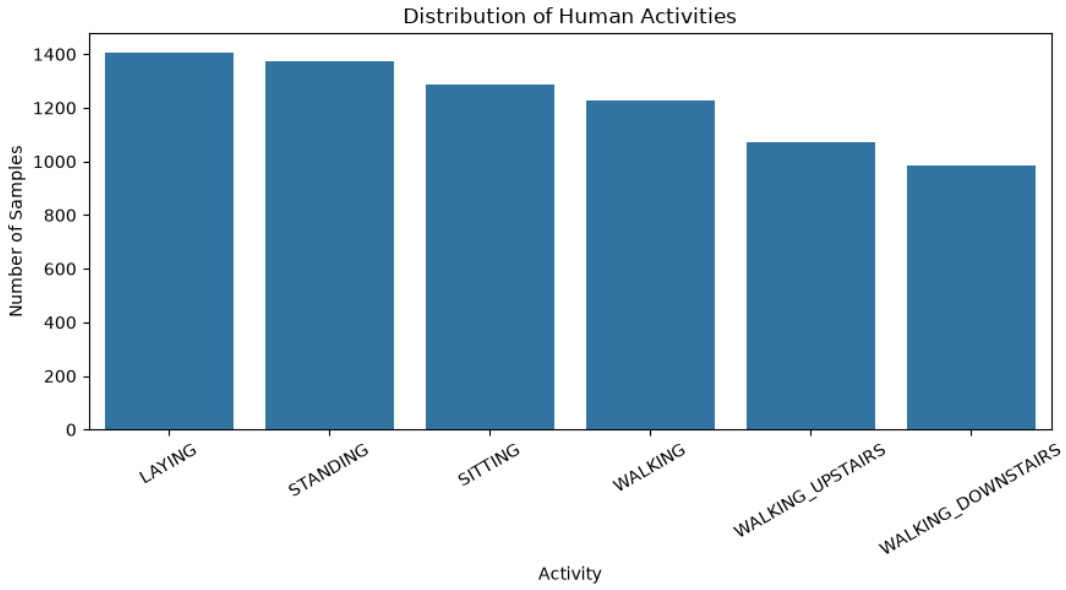
WEEK - 7

d. EDA univariate analysis

```
activity_counts = pd.Series(y_train).map(activity_names).value_counts()
print(activity_counts)

plt.figure(figsize=(9, 5))
sns.barplot(x=activity_counts.index, y=activity_counts.values)
plt.title("Distribution of Human Activities")
plt.xlabel("Activity"); plt.ylabel("Number of Samples")
plt.xticks(rotation=30); plt.tight_layout(); plt.show()
```

```
LAYING                1407
STANDING              1374
SITTING               1286
WALKING               1226
WALKING_UPSTAIRS      1073
WALKING_DOWNSTAIRS     986
Name: count, dtype: int64
```



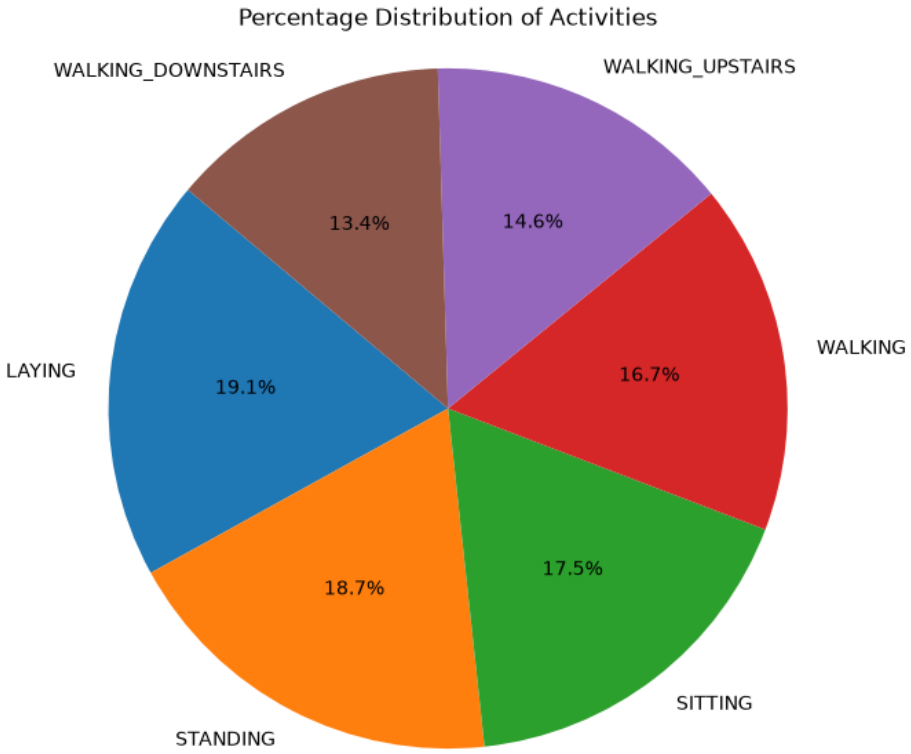

```
plt.figure(figsize=(7, 7))
plt.pie(activity_counts.values, labels=activity_counts.index, autopct="%1.1f%%",
startangle=140)
plt.title("Percentage Distribution of Activities")
plt.axis("equal"); plt.show()

print("Largest / smallest class ratio:", round(activity_counts.max() / activity_counts.min(),
2))
```

VNRVJIET

NAME OF LAB: _____ **WEEK:** _____

EXPERIMENT NAME:



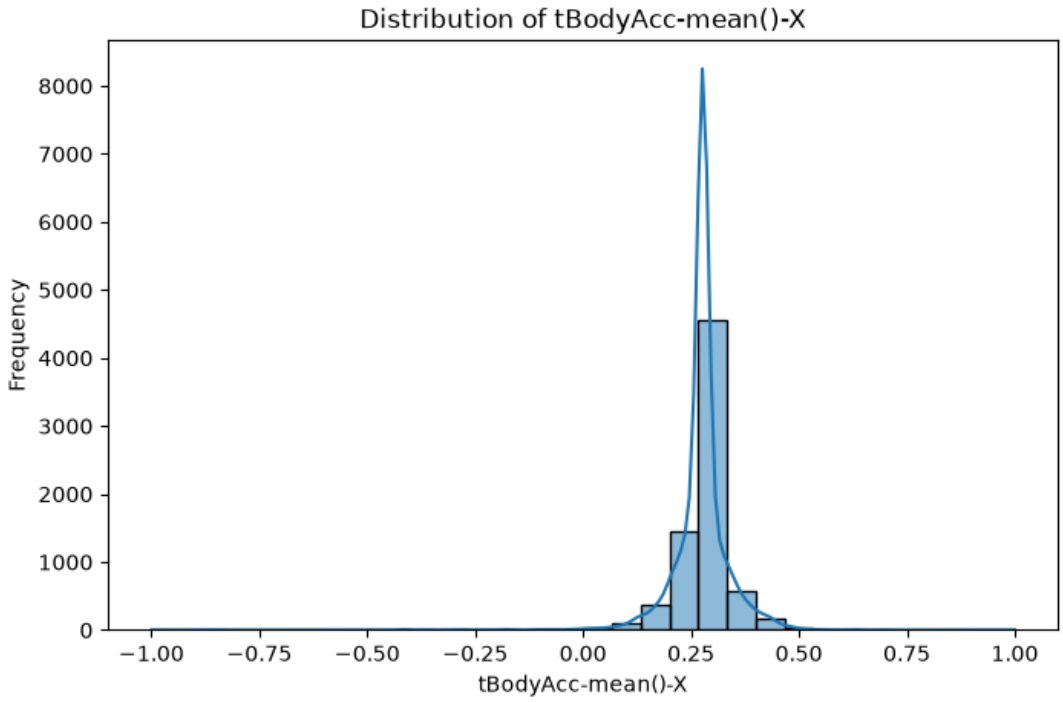

```
Largest / smallest class ratio: 1.43
```

```
feature = feature_names[0]
plt.figure(figsize=(8, 5))
sns.histplot(X_train[feature], bins=30, kde=True)
plt.title(f"Distribution of {feature}")
plt.xlabel(feature); plt.ylabel("Frequency"); plt.show()
print(X_train[feature].describe().round(4))
```

VNRVJIET

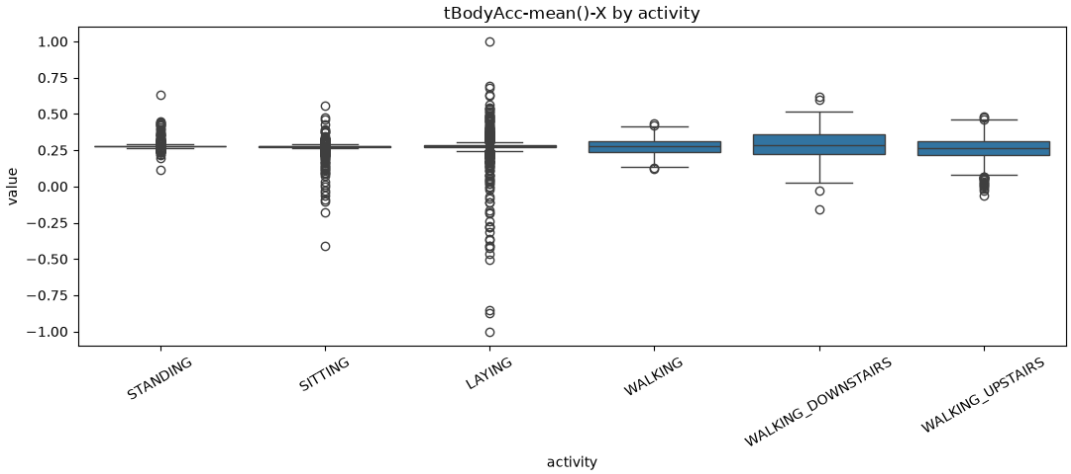
NAME OF LAB: _____ WEEK: _____

EXPERIMENT NAME:

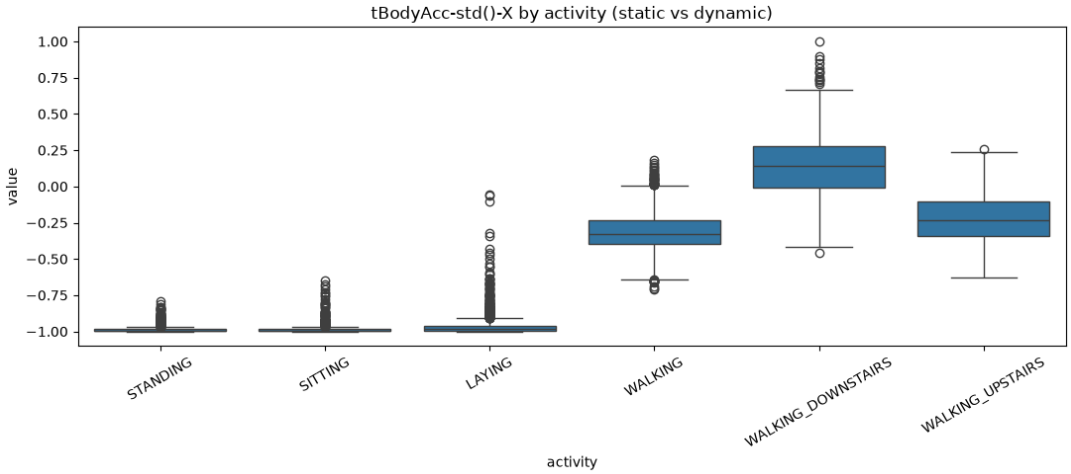



```
count    7352.0000
mean        0.2745
std         0.0703
min        -1.0000
25%         0.2630
50%         0.2772
75%         0.2885
max         1.0000
Name: tBodyAcc-mean()-X, dtype: float64
```

```
plt.figure(figsize=(11, 5))
tmp = pd.DataFrame({"value": X_train[feature].values, "activity":
pd.Series(y_train).map(activity_names)})
sns.boxplot(data=tmp, x="activity", y="value")
plt.title(f"{feature} by activity"); plt.xticks(rotation=30); plt.tight_layout(); plt.show()
```




```
# A feature that usually separates static vs moving activities very well: the std of body acceleration
std_feature = "tBodyAcc-std()-X"
plt.figure(figsize=(11, 5))
tmp = pd.DataFrame({"value": X_train[std_feature].values, "activity":
pd.Series(y_train).map(activity_names)})
sns.boxplot(data=tmp, x="activity", y="value")
plt.title(f"{std_feature} by activity (static vs dynamic)"); plt.xticks(rotation=30);
plt.tight_layout(); plt.show()
```



VNRVJIET

NAME OF LAB: _____ WEEK: _____

EXPERIMENT NAME:

WEEK - 8

e. Classical ML models

```
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC

models = {
    "Logistic Regression": LogisticRegression(max_iter=2000),
    "Random Forest": RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1),
    "SVM": SVC(kernel="rbf"),
}

results = []
for name, model in models.items():
    print("\nTraining:", name)
    model.fit(X_train_scaled, y_train_encoded)
    predictions = model.predict(X_test_scaled)

    results.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test_encoded, predictions),
        "Precision": precision_score(y_test_encoded, predictions, average="weighted"),
        "Recall": recall_score(y_test_encoded, predictions, average="weighted"),
        "F1 Score": f1_score(y_test_encoded, predictions, average="weighted"),
    })

    print(classification_report(
        y_test_encoded, predictions,
        labels=np.arange(6),
        target_names=[activity_names[i] for i in range(1, 7)],
        zero_division=0))
```

```
Training: Logistic Regression
                    precision    recall  f1-score   support

           WALKING       0.94      0.99      0.97       496
  WALKING_UPSTAIRS       0.96      0.95      0.95       471
WALKING_DOWNSTAIRS       0.99      0.94      0.96       420
           SITTING       0.97      0.88      0.92       491
          STANDING       0.89      0.98      0.93       532
            LAYING       1.00      0.99      0.99       537

          accuracy                           0.95      2947
         macro avg       0.96      0.95      0.95      2947
      weighted avg       0.96      0.95      0.95      2947


Training: Random Forest
                    precision    recall  f1-score   support

           WALKING       0.89      0.97      0.93       496
  WALKING_UPSTAIRS       0.90      0.89      0.90       471
```

```
WALKING_DOWNSTAIRS       0.96      0.87      0.91       420
           SITTING       0.91      0.90      0.90       491
          STANDING       0.91      0.92      0.91       532
            LAYING       1.00      1.00      1.00       537

          accuracy                           0.93      2947
         macro avg       0.93      0.92      0.93      2947
      weighted avg       0.93      0.93      0.93      2947


Training: SVM
                    precision    recall  f1-score   support

           WALKING       0.96      0.97      0.97       496
  WALKING_UPSTAIRS       0.93      0.97      0.95       471
WALKING_DOWNSTAIRS       0.98      0.92      0.95       420
           SITTING       0.94      0.90      0.92       491
          STANDING       0.92      0.95      0.93       532
            LAYING       1.00      1.00      1.00       537

          accuracy                           0.95      2947
         macro avg       0.95      0.95      0.95      2947
      weighted avg       0.95      0.95      0.95      2947
```

```
results_df = pd.DataFrame(results)
display(results_df.sort_values("Accuracy", ascending=False).round(4))
```

```
                Model  Accuracy  Precision  Recall  F1 Score
0  Logistic Regression    0.9545     0.9565  0.9545    0.9545
2                  SVM    0.9522     0.9526  0.9522    0.9521
1        Random Forest    0.9267     0.9278  0.9267    0.9266
```

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Conv1D, MaxPooling1D, Flatten, Dense, Dropout, LSTM
from tensorflow.keras.callbacks import EarlyStopping
tf.random.set_seed(42)

X_tr, X_val, y_tr, y_val = train_test_split(
    X_train_scaled, y_train_encoded, test_size=0.1, random_state=42, stratify=y_train_encoded)

X_tr_dl = X_tr.astype("float32").reshape(X_tr.shape[0], X_tr.shape[1], 1)
X_val_dl = X_val.astype("float32").reshape(X_val.shape[0], X_val.shape[1], 1)
X_test_dl = X_test_scaled.astype("float32").reshape(X_test_scaled.shape[0],
X_test_scaled.shape[1], 1)

print("Training:", X_tr_dl.shape)
print("Validation:", X_val_dl.shape)
print("Testing:", X_test_dl.shape)
```

```
Training: (6616, 561, 1)
Validation: (736, 561, 1)
Testing: (2947, 561, 1)
```

```
cnn_model = Sequential([
```

```
    Input(shape=(561, 1)),
    Conv1D(64, 3, activation="relu"),
    MaxPooling1D(2),
    Dropout(0.3),
    Conv1D(128, 3, activation="relu"),
    MaxPooling1D(2),
    Flatten(),
    Dense(128, activation="relu"),
    Dropout(0.3),
    Dense(6, activation="softmax"),
])
cnn_model.compile(optimizer="adam", loss="sparse_categorical_crossentropy",
metrics=["accuracy"])
cnn_model.summary()

early_stop = EarlyStopping(monitor="val_loss", patience=3, restore_best_weights=True)

cnn_history = cnn_model.fit(X_tr_dl, y_tr, validation_data=(X_val_dl, y_val),
                            epochs=15, batch_size=64, callbacks=[early_stop], verbose=2)

cnn_loss, cnn_accuracy = cnn_model.evaluate(X_test_dl, y_test_encoded, verbose=0)
print("CNN Test Accuracy:", round(cnn_accuracy, 4))
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.
```

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv1d (Conv1D)	(None, 559, 64)	256
max_pooling1d (MaxPooling1D)	(None, 279, 64)	0
dropout (Dropout)	(None, 279, 64)	0
conv1d_1 (Conv1D)	(None, 277, 128)	24,704
max_pooling1d_1 (MaxPooling1D)	(None, 138, 128)	0
flatten (Flatten)	(None, 17664)	0
dense (Dense)	(None, 128)	2,261,120
dropout_1 (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 6)	774

```
 Total params: 2,286,854 (8.72 MB)
```

```
 Trainable params: 2,286,854 (8.72 MB)
```

```
 Non-trainable params: 0 (0.00 B)
```

```
Epoch 1/15
104/104 - 7s - 66ms/step - accuracy: 0.7937 - loss: 0.5052 - val_accuracy: 0.9484 - val_loss:
0.1472
Epoch 2/15
104/104 - 4s - 36ms/step - accuracy: 0.9414 - loss: 0.1564 - val_accuracy: 0.9701 - val_loss:
0.0836
Epoch 3/15
104/104 - 4s - 40ms/step - accuracy: 0.9606 - loss: 0.1087 - val_accuracy: 0.9620 - val_loss:
0.0941
Epoch 4/15
104/104 - 4s - 38ms/step - accuracy: 0.9681 - loss: 0.0826 - val_accuracy: 0.9715 - val_loss:
0.0792
Epoch 5/15
104/104 - 3s - 33ms/step - accuracy: 0.9772 - loss: 0.0634 - val_accuracy: 0.9810 - val_loss:
0.0528
Epoch 6/15
104/104 - 3s - 31ms/step - accuracy: 0.9794 - loss: 0.0527 - val_accuracy: 0.9796 - val_loss:
0.0527
Epoch 7/15
104/104 - 3s - 31ms/step - accuracy: 0.9813 - loss: 0.0483 - val_accuracy: 0.9837 - val_loss:
0.0438
Epoch 8/15
104/104 - 3s - 32ms/step - accuracy: 0.9850 - loss: 0.0416 - val_accuracy: 0.9878 - val_loss:
0.0385
Epoch 9/15
104/104 - 3s - 32ms/step - accuracy: 0.9846 - loss: 0.0436 - val_accuracy: 0.9823 - val_loss:
0.0517
Epoch 10/15
104/104 - 3s - 31ms/step - accuracy: 0.9902 - loss: 0.0278 - val_accuracy: 0.9742 - val_loss:
0.0612
Epoch 11/15
104/104 - 3s - 32ms/step - accuracy: 0.9893 - loss: 0.0313 - val_accuracy: 0.9878 - val_loss:
0.0323
Epoch 12/15
104/104 - 3s - 32ms/step - accuracy: 0.9903 - loss: 0.0273 - val_accuracy: 0.9851 - val_loss:
0.0317
Epoch 13/15
104/104 - 3s - 32ms/step - accuracy: 0.9912 - loss: 0.0261 - val_accuracy: 0.9905 - val_loss:
0.0314
Epoch 14/15
104/104 - 3s - 32ms/step - accuracy: 0.9935 - loss: 0.0182 - val_accuracy: 0.9796 - val_loss:
0.0473
Epoch 15/15
104/104 - 3s - 32ms/step - accuracy: 0.9911 - loss: 0.0241 - val_accuracy: 0.9905 - val_loss:
0.0336
CNN Test Accuracy: 0.9589
```

```
lstm_model = Sequential([
    Input(shape=(561, 1)),
    LSTM(64),
    Dropout(0.3),
    Dense(64, activation="relu"),
    Dense(6, activation="softmax"),
])
lstm_model.compile(optimizer="adam", loss="sparse_categorical_crossentropy",
metrics=["accuracy"])
```

```
lstm_history = lstm_model.fit(X_tr_dl, y_tr, validation_data=(X_val_dl, y_val),
                              epochs=15, batch_size=64, callbacks=[early_stop], verbose=2)

lstm_loss, lstm_accuracy = lstm_model.evaluate(X_test_dl, y_test_encoded, verbose=0)
print("LSTM Test Accuracy:", round(lstm_accuracy, 4))
```

```
Epoch 1/15
104/104 - 31s - 300ms/step - accuracy: 0.4285 - loss: 1.2357 - val_accuracy: 0.6223 - val_loss:
0.8923
Epoch 2/15
104/104 - 29s - 283ms/step - accuracy: 0.5913 - loss: 0.8625 - val_accuracy: 0.6658 - val_loss:
0.7367
Epoch 3/15
104/104 - 27s - 258ms/step - accuracy: 0.6208 - loss: 0.9051 - val_accuracy: 0.6454 - val_loss:
0.8343
Epoch 4/15
104/104 - 21s - 202ms/step - accuracy: 0.6400 - loss: 0.8280 - val_accuracy: 0.6304 - val_loss:
0.7753
Epoch 5/15
104/104 - 24s - 231ms/step - accuracy: 0.6666 - loss: 0.7197 - val_accuracy: 0.7106 - val_loss:
0.6485
Epoch 6/15
104/104 - 22s - 208ms/step - accuracy: 0.6982 - loss: 0.6601 - val_accuracy: 0.7215 - val_loss:
0.6139
Epoch 7/15
104/104 - 24s - 230ms/step - accuracy: 0.7142 - loss: 0.6244 - val_accuracy: 0.7378 - val_loss:
0.5970
Epoch 8/15
104/104 - 23s - 223ms/step - accuracy: 0.7313 - loss: 0.5942 - val_accuracy: 0.7269 - val_loss:
0.5752
Epoch 9/15
104/104 - 23s - 221ms/step - accuracy: 0.7320 - loss: 0.5779 - val_accuracy: 0.7514 - val_loss:
0.5497
Epoch 10/15
104/104 - 24s - 226ms/step - accuracy: 0.7527 - loss: 0.5536 - val_accuracy: 0.7677 - val_loss:
0.4996
Epoch 11/15
104/104 - 25s - 238ms/step - accuracy: 0.7851 - loss: 0.5086 - val_accuracy: 0.8043 - val_loss:
0.4580
Epoch 12/15
104/104 - 25s - 237ms/step - accuracy: 0.8018 - loss: 0.4749 - val_accuracy: 0.8111 - val_loss:
0.4439
Epoch 13/15
104/104 - 23s - 225ms/step - accuracy: 0.8141 - loss: 0.4498 - val_accuracy: 0.8261 - val_loss:
0.4305
Epoch 14/15
104/104 - 19s - 184ms/step - accuracy: 0.8195 - loss: 0.4400 - val_accuracy: 0.8247 - val_loss:
0.4147
Epoch 15/15
104/104 - 21s - 201ms/step - accuracy: 0.8201 - loss: 0.4244 - val_accuracy: 0.8234 - val_loss:
0.4056
LSTM Test Accuracy: 0.794
```

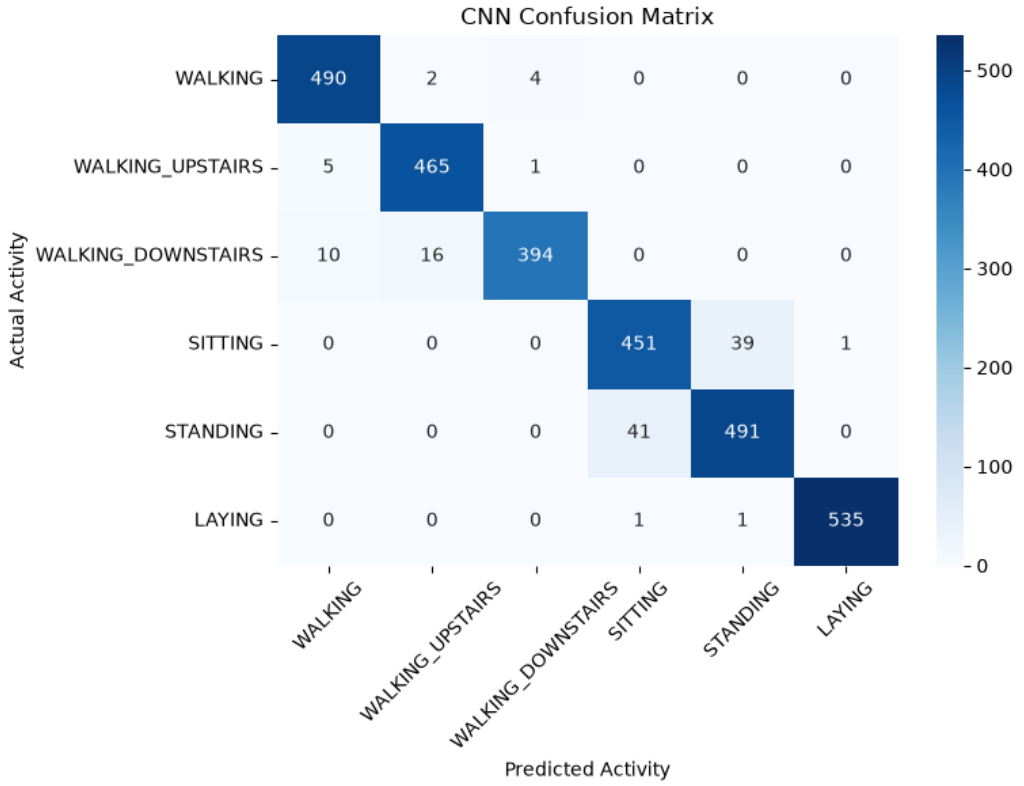
```
cnn_predictions = np.argmax(cnn_model.predict(X_test_dl, verbose=0), axis=1)
lstm_predictions = np.argmax(lstm_model.predict(X_test_dl, verbose=0), axis=1)
```

```
deep_results = pd.DataFrame([
    {"Model": "CNN", "Accuracy": accuracy_score(y_test_encoded, cnn_predictions)},
    {"Model": "LSTM", "Accuracy": accuracy_score(y_test_encoded, lstm_predictions)},
])
display(deep_results.round(4))
```

```
  Model  Accuracy
0   CNN    0.9589
1  LSTM    0.7940
```

```
cm = confusion_matrix(y_test_encoded, cnn_predictions)
labels = [activity_names[i] for i in range(1, 7)]

plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=labels, yticklabels=labels)
plt.title("CNN Confusion Matrix"); plt.xlabel("Predicted Activity"); plt.ylabel("Actual Activity")
plt.xticks(rotation=45); plt.tight_layout(); plt.show()
```




```
# Overall comparison: classical + deep models in one table
all_results = pd.concat([results_df[["Model", "Accuracy"]], deep_results], ignore_index=True)
display(all_results.sort_values("Accuracy", ascending=False).round(4))
```

```
                Model  Accuracy
3                 CNN    0.9589
0  Logistic Regression    0.9545
```

```
2                 SVM    0.9522
1       Random Forest    0.9267
4                LSTM    0.7940
```

VNRVJIET

NAME OF LAB: _____ WEEK: _____

EXPERIMENT NAME:

WEEK - 9

Title of the Experiment: Quora question pair similarity problem

Objective of the experiment:

Understand the importance of duplicate question detection in NLP applications. Perform Exploratory Data Analysis (EDA) on the Quora Question Pairs dataset. Extract key text-based features (TF-IDF, Word2Vec, Sentence Embeddings). Implement traditional ML models (Logistic Regression, SVM, Random Forest). Build Deep Learning models (LSTMs, Bi-LSTMs, BERT, etc.) for text similarity. Evaluate performance using precision, recall, F1-score, and accuracy.

a. Business real world problem definition

The objective is to identify whether two questions posted on a question-answering platform express the same intent. Duplicate detection can help reduce repeated questions, improve search, and direct users to existing answers.

b. Business objectives and constraints

- Detect duplicate question pairs accurately.
- Reduce unnecessary repeated questions.
- Minimize false positives (different questions incorrectly marked as duplicates) and false negatives (duplicates missed).
- Provide duplicate probabilities when possible, allowing the decision threshold to be adjusted.
- Handle large volumes of text within reasonable memory and processing time.

c. Mapping to an ML problem: data overview

```
import re
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split

df = pd.read_csv("data/quora/train.csv")
```

```
print("Dataset shape:", df.shape)
print("\nColumns:", df.columns.tolist())
print("\nMissing values:\n", df[["question1", "question2", "is_duplicate"]].isnull().sum())

display(df.head())
```

```
print("\nTarget distribution:")
print(df["is_duplicate"].value_counts())
```

```
Dataset shape: (404276, 6)

Columns: ['id', 'qid1', 'qid2', 'question1', 'question2', 'is_duplicate']

Missing values:
 question1       0
question2       0
is_duplicate    0
dtype: int64
```

```
   id  qid1    qid2                                          question1  \
0   0     1  290449  How is the life of a math student? Could you d...
1   1     2  290450                How do I control my horny emotions?
2   2     3  290451       What causes stool color to change to yellow?
3   3     4  290452                          What can one do after MBBS?
4   4     5  290453  Where can I find a power outlet for my laptop ...

                                          question2  is_duplicate
0  Which level of prepration is enough for the ex...             0
1                How do you control your horniness?             1
2  What can cause stool to come out as little balls?             0
3                      What do i do after my MBBS ?             1
4  Would a second airport in Sydney, Australia be...             0
```

```
Target distribution:
is_duplicate
0    255013
1    149263
Name: count, dtype: int64
```

```
for label in (1, 0):
    print("=== is_duplicate =", label)
    for _, r in df[df["is_duplicate"] == label].sample(3, random_state=1).iterrows():
        print("  Q1:", r["question1"]); print("  Q2:", r["question2"]); print()
```

```
=== is_duplicate = 1
  Q1: Why does the handlebar of the cyclist's bicycle so low?
  Q2: Why is the handlebar of the cyclist's bicycle so low?

  Q1: What are signs my guy friend is flirting with me?
  Q2: What are signs that your guy friend/ buddy is flirting with you (we are both in our mid
twenties)?

  Q1: How can we reduce humidity in our homes? Any tips?
  Q2: How can I reduce humidity in a room?

=== is_duplicate = 0
  Q1: Who would win in a battle between a wolf and a sun bear?
  Q2: Why does all Indian players is with beard in this newzealand test series?

  Q1: When will the academic year starts in IIM?
  Q2: Is IIM and FMS possible with a low grade?

  Q1: What one dead person would you most like to have dinner with, if it were possible?
```

VNRVJIET

NAME OF LAB: _____ WEEK: _____

EXPERIMENT NAME:

Q2: If you could have dinner with one famous person, dead or alive, who would it be, and why?

WEEK - 10

d. Mapping to an ML problem and performance measures

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, log_loss

y_true = np.array([1, 1, 1, 0, 0, 0, 0, 0])
y_pred = np.array([1, 1, 0, 1, 0, 0, 0, 0])      # 2 TP, 1 FN, 1 FP, 4 TN
y_prob = np.array([.9, .8, .4, .6, .2, .1, .3, .2])

print("Accuracy :", accuracy_score(y_true, y_pred), "  (6 of 8 correct)")
print("Precision:", round(precision_score(y_true, y_pred), 3), "  = TP/(TP+FP) = 2/3")
print("Recall   :", round(recall_score(y_true, y_pred), 3), "  = TP/(TP+FN) = 2/3")
print("F1       :", round(f1_score(y_true, y_pred), 3))
print("Log-loss :", round(log_loss(y_true, y_prob), 3))
```

```
Accuracy : 0.75   (6 of 8 correct)
Precision: 0.667   = TP/(TP+FP) = 2/3
Recall   : 0.667   = TP/(TP+FN) = 2/3
F1       : 0.667
Log-loss : 0.384
```

e. Mapping to an ML problem: train-test split

```
df = df.dropna(subset=["is_duplicate"]).copy()
df["question1"] = df["question1"].fillna("")
df["question2"] = df["question2"].fillna("")
df["is_duplicate"] = df["is_duplicate"].astype(int)

train_df, test_df = train_test_split(df, test_size=0.2, random_state=42,
stratify=df["is_duplicate"])

print("Training shape:", train_df.shape)
print("Testing shape:", test_df.shape)
print("\nTraining label distribution:")
print(train_df["is_duplicate"].value_counts(normalize=True))
print("\nTesting label distribution:")
print(test_df["is_duplicate"].value_counts(normalize=True))
```

```
Training shape: (323420, 6)
Testing shape: (80856, 6)

Training label distribution:
is_duplicate
0    0.63079
1    0.36921
Name: proportion, dtype: float64

Testing label distribution:
is_duplicate
0    0.630788
1    0.369212
Name: proportion, dtype: float64
```

f. EDA basic statistics, feature extraction

```
print("Duplicate ratio:", df["is_duplicate"].mean())
print("\nDataset statistics:")
print(df[["is_duplicate"]].describe())

df["q1_length"] = df["question1"].str.len()
df["q2_length"] = df["question2"].str.len()
df["length_difference"] = (df["q1_length"] - df["q2_length"]).abs()
df["q1_word_count"] = df["question1"].str.split().str.len()
df["q2_word_count"] = df["question2"].str.split().str.len()

display(df[["question1", "question2", "is_duplicate", "q1_length", "q2_length",
"length_difference"]].head())
```

```
Duplicate ratio: 0.36921063827682077

Dataset statistics:
        is_duplicate
count  404276.000000
mean        0.369211
std         0.482592
min         0.000000
25%         0.000000
50%         0.000000
75%         1.000000
max         1.000000
```

```
                                          question1  \
0  How is the life of a math student? Could you d...
1                How do I control my horny emotions?
2       What causes stool color to change to yellow?
3                          What can one do after MBBS?
4  Where can I find a power outlet for my laptop ...

                                          question2  is_duplicate  q1_length  \
0  Which level of prepration is enough for the ex...             0         75
1                 How do you control your horniness?             1         35
2  What can cause stool to come out as little balls?             0         44
3                        What do i do after my MBBS ?             1         27
4  Would a second airport in Sydney, Australia be...             0         67

   q2_length  length_difference
0         55                 20
1         34                  1
2         49                  5
3         28                  1
4        121                 54
```

```
print(df[["q1_length", "q2_length", "q1_word_count", "q2_word_count",
"length_difference"]].describe().round(1))
print("\nMean length difference by class:")
print(df.groupby("is_duplicate")["length_difference"].mean().round(2))
```

```
       q1_length  q2_length  q1_word_count  q2_word_count  length_difference
count   404276.0   404276.0       404276.0       404276.0           404276.0
```

```
mean        59.6       60.2          10.9          11.2          20.2
std         30.0       33.9           5.4           6.3          25.6
min          1.0        1.0           1.0           1.0           0.0
25%         39.0       39.0           7.0           7.0           5.0
50%         52.0       51.0          10.0          10.0          12.0
75%         72.0       72.0          13.0          13.0          26.0
max        627.0     1175.0         125.0         237.0        1080.0

Mean length difference by class:
is_duplicate
0    24.33
1    13.16
Name: length_difference, dtype: float64
```

VNRVJIET

NAME OF LAB: _____ WEEK: _____

EXPERIMENT NAME:

WEEK - 11

g. EDA text processing, advanced feature extraction

```
import re

def clean_text(text):
    text = str(text).lower()
    text = re.sub(r"[^a-z0-9\s]", " ", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text

q1, q2 = "How do I learn Python quickly?", "What is the fastest way to learn Python?"
print("Cleaned Q1:", clean_text(q1))
print("Cleaned Q2:", clean_text(q2))
```

```
Cleaned Q1: how do i learn python quickly
Cleaned Q2: what is the fastest way to learn python
```

```
def pair_features(q1, q2):
    words1 = set(clean_text(q1).split())
    words2 = set(clean_text(q2).split())
    common = len(words1 & words2)
    total = len(words1 | words2)
    return {
        "q1_length": len(str(q1)),
        "q2_length": len(str(q2)),
        "length_difference": abs(len(str(q1)) - len(str(q2))),
        "common_words": common,
        "jaccard_similarity": common / total if total else 0,
    }

print(pair_features(q1, q2))
```

```
{'q1_length': 30, 'q2_length': 40, 'length_difference': 10, 'common_words': 2,
'jaccard_similarity': 0.16666666666666666}
```

```
train_df = train_df.copy()
test_df = test_df.copy()
train_df["question1_clean"] = train_df["question1"].apply(clean_text)
train_df["question2_clean"] = train_df["question2"].apply(clean_text)
test_df["question1_clean"] = test_df["question1"].apply(clean_text)
test_df["question2_clean"] = test_df["question2"].apply(clean_text)

display(train_df[["question1_clean", "question2_clean", "is_duplicate"]].head())
```

```
                                    question1_clean  \
268624  my dell inspiron 15r 5537 make noice tuck tuck...
385786  why is bill gates net worth still rising when ...
257720  how do bernie sanders and donald trump differ ...
324425                          what s the meaning of life
269584  what is the best open source php form builder ...

                                    question2_clean  is_duplicate
```

```
268624  i was set up on a date unknowingly i ve always...           0
385786  how come bill gates is still the richest man i...           1
257720  what are the differences between the global tr...           1
324425                what is the meaning of life to you            1
269584  what are the ways to insert an image path into...           0
```

```
# The record builds a DataFrame from a list of pd.Series, which is slow; returning plain dicts
gives the same table much faster.
train_pair_features = pd.DataFrame([pair_features(a, b) for a, b in zip(train_df["question1"],
train_df["question2"])])
test_pair_features = pd.DataFrame([pair_features(a, b) for a, b in zip(test_df["question1"],
test_df["question2"])])

display(train_pair_features.head())
print("Train features:", train_pair_features.shape, "| Test features:",
test_pair_features.shape)
```

```
   q1_length  q2_length  length_difference  common_words  jaccard_similarity
0         69        148                 79             2            0.057143
1         72        168                 96             9            0.264706
2         55         94                 39             6            0.333333
3         27         35                  8             5            0.555556
4        143        141                  2             8            0.190476
```

```
Train features: (323420, 5) | Test features: (80856, 5)
```

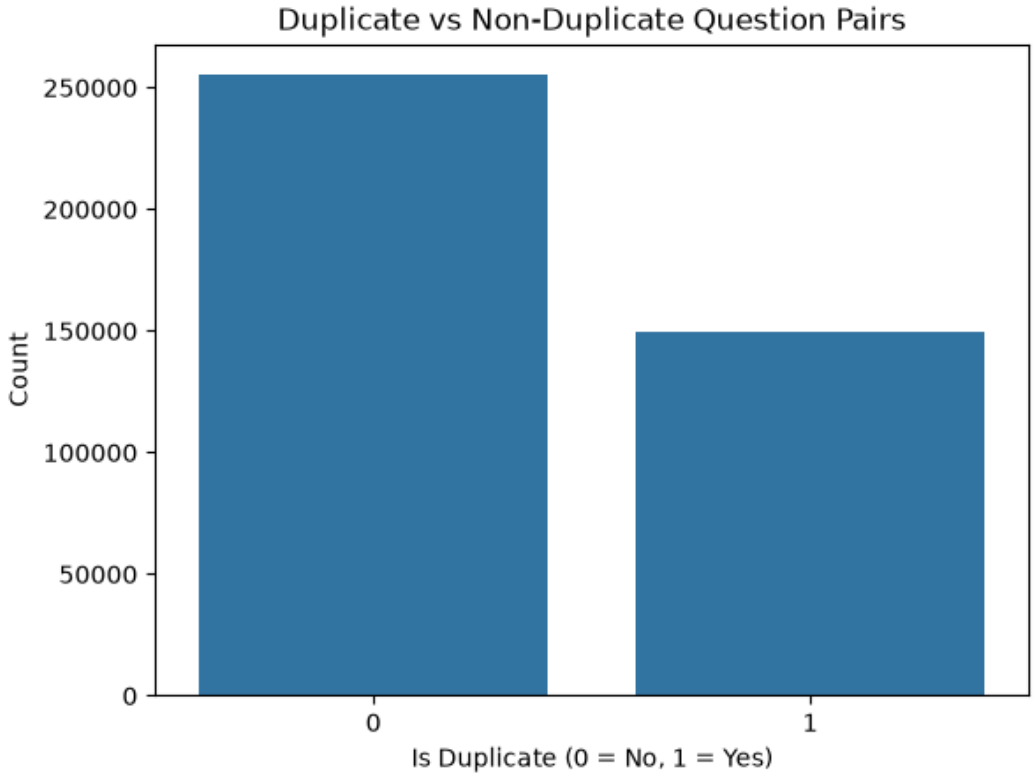
h. EDA feature analysis, data visualization

```
sns.countplot(data=df, x="is_duplicate")
plt.title("Duplicate vs Non-Duplicate Question Pairs")
plt.xlabel("Is Duplicate (0 = No, 1 = Yes)"); plt.ylabel("Count"); plt.show()
```

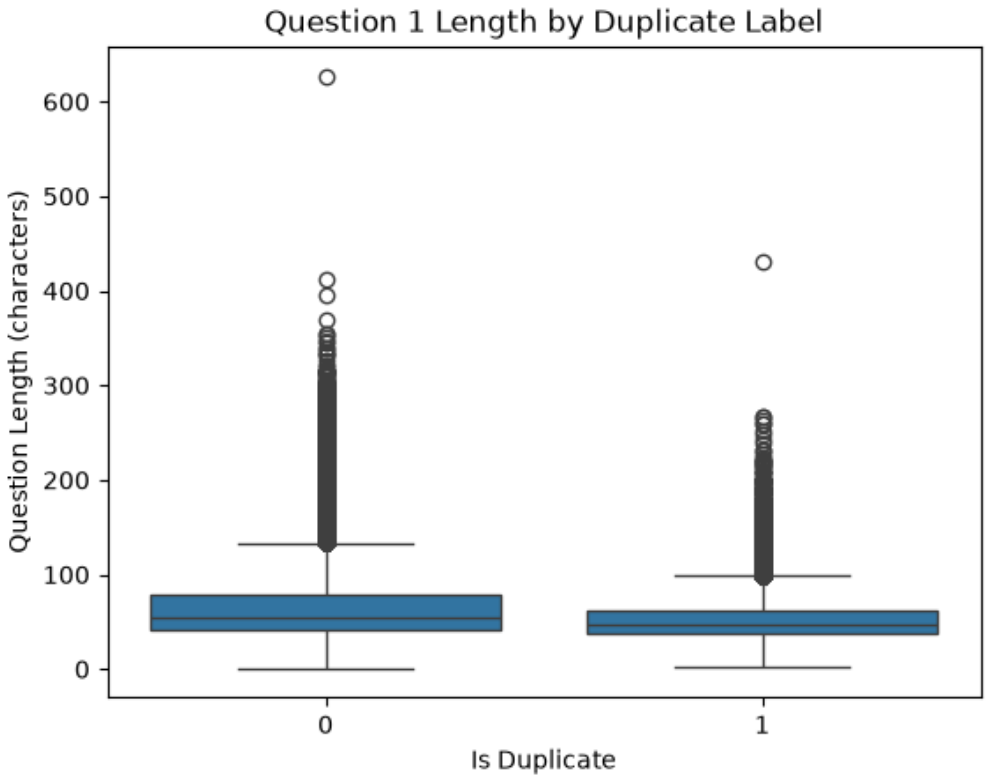
VNRVJIET

NAME OF LAB: _____ WEEK: _____

EXPERIMENT NAME:



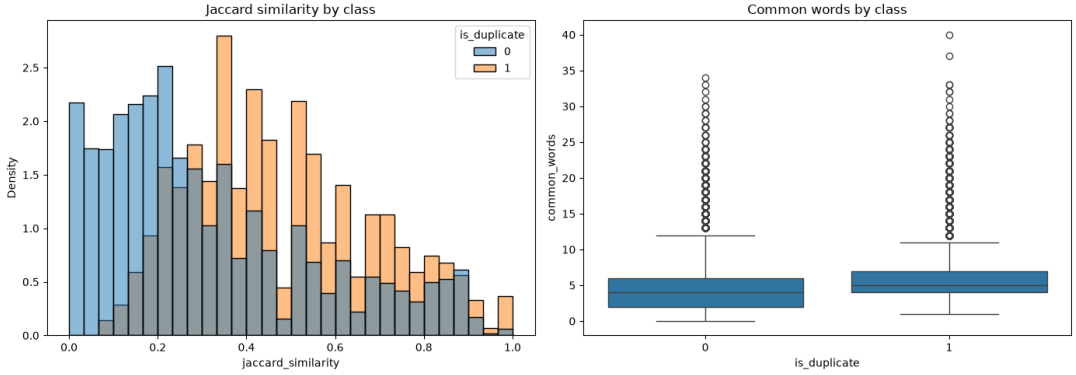

```
sns.boxplot(data=df, x="is_duplicate", y="q1_length")
plt.title("Question 1 Length by Duplicate Label")
plt.xlabel("Is Duplicate"); plt.ylabel("Question Length (characters)"); plt.show()
```




```
plot_df = train_pair_features.copy()
plot_df["is_duplicate"] = train_df["is_duplicate"].values

fig, axs = plt.subplots(1, 2, figsize=(14, 5))
sns.histplot(data=plot_df, x="jaccard_similarity", hue="is_duplicate", bins=30,
stat="density", common_norm=False, ax=axs[0])
axs[0].set_title("Jaccard similarity by class")
sns.boxplot(data=plot_df, x="is_duplicate", y="common_words", ax=axs[1])
axs[1].set_title("Common words by class")
plt.tight_layout(); plt.show()

print(plot_df.groupby("is_duplicate")[["common_words", "jaccard_similarity",
"length_difference"]].mean().round(3))
```



```
              common_words  jaccard_similarity  length_difference
is_duplicate
0                    4.529               0.307             24.337
1                    5.860               0.472             13.169
```

VNRVJIET

NAME OF LAB: _____ WEEK: _____

EXPERIMENT NAME:

WEEK - 12

i. EDA TF-IDF weighting, Word2Vec, factorization

```
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer(max_features=50000, ngram_range=(1, 2), min_df=2,
sublinear_tf=True)
train_questions = pd.concat([train_df["question1_clean"], train_df["question2_clean"]])
vectorizer.fit(train_questions)

q1_train_tfidf = vectorizer.transform(train_df["question1_clean"])
q2_train_tfidf = vectorizer.transform(train_df["question2_clean"])
q1_test_tfidf = vectorizer.transform(test_df["question1_clean"])
q2_test_tfidf = vectorizer.transform(test_df["question2_clean"])

print("TF-IDF shape:", q1_train_tfidf.shape)
terms = vectorizer.get_feature_names_out()
print("Sample vocabulary terms:", list(terms[:5]), "...", list(terms[20000:20005]))
```

```
TF-IDF shape: (323420, 50000)
Sample vocabulary terms: ['00', '000', '000 000', '000 dollars', '000 in'] ... ['hurt so', 'hurt
the', 'hurt to', 'hurt when', 'hurt you']
```

```
from scipy.sparse import hstack

num_cols = ["length_difference", "common_words", "jaccard_similarity"]

X_train_tfidf = hstack([
    abs(q1_train_tfidf - q2_train_tfidf),
    q1_train_tfidf.multiply(q2_train_tfidf),
    train_pair_features[num_cols].to_numpy(),
]).tocsr()

X_test_tfidf = hstack([
    abs(q1_test_tfidf - q2_test_tfidf),
    q1_test_tfidf.multiply(q2_test_tfidf),
    test_pair_features[num_cols].to_numpy(),
]).tocsr()

y_train = train_df["is_duplicate"].to_numpy()
y_test = test_df["is_duplicate"].to_numpy()

print("Training feature matrix:", X_train_tfidf.shape)
print("Testing feature matrix:", X_test_tfidf.shape)
print("Sparsity: only", round(X_train_tfidf.nnz / np.prod(X_train_tfidf.shape) * 100, 4), "%
of entries are non-zero")
```

```
Training feature matrix: (323420, 100003)
Testing feature matrix: (80856, 100003)
Sparsity: only -0.5532 % of entries are non-zero
```

```
from sklearn.decomposition import TruncatedSVD
```

```
sample = q1_train_tfidf[:50000]
svd = TruncatedSVD(n_components=50, random_state=42)
reduced = svd.fit_transform(sample)
print("Original:", sample.shape, "-> reduced:", reduced.shape)
print("Variance explained by 50 components:", round(svd.explained_variance_ratio_.sum() * 100,
1), "%")
```

```
Original: (50000, 50000) -> reduced: (50000, 50)
Variance explained by 50 components: 10.4 %
```

j. ML models loading, random model

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, log_loss

def evaluate_model(y_true, y_pred, y_prob=None):
    print("Accuracy:", round(accuracy_score(y_true, y_pred), 4))
    print("Precision:", round(precision_score(y_true, y_pred, zero_division=0), 4))
    print("Recall:", round(recall_score(y_true, y_pred, zero_division=0), 4))
    print("F1 Score:", round(f1_score(y_true, y_pred, zero_division=0), 4))
    if y_prob is not None:
        print("Log Loss:", round(log_loss(y_true, y_prob), 4))

rng = np.random.default_rng(42)
random_predictions = rng.integers(0, 2, size=len(y_test))

print("Random Model Results:")
evaluate_model(y_test, random_predictions)
```

```
Random Model Results:
Accuracy: 0.5008
Precision: 0.3702
Recall: 0.5021
F1 Score: 0.4262
```

```
from sklearn.linear_model import LogisticRegression

lr_model = LogisticRegression(max_iter=1000, solver="liblinear")
lr_model.fit(X_train_tfidf, y_train)
lr_predictions = lr_model.predict(X_test_tfidf)
lr_probabilities = lr_model.predict_proba(X_test_tfidf)[:, 1]

print("Logistic Regression Results:")
evaluate_model(y_test, lr_predictions, lr_probabilities)
```

```
Logistic Regression Results:
Accuracy: 0.8168
Precision: 0.7726
Recall: 0.714
F1 Score: 0.7421
Log Loss: 0.3953
```

```
from sklearn.svm import LinearSVC

svm_model = LinearSVC(random_state=42)
```

```
svm_model.fit(X_train_tfidf, y_train)
svm_predictions = svm_model.predict(X_test_tfidf)

print("Linear SVM Results:")
evaluate_model(y_test, svm_predictions)
```

```
Linear SVM Results:
Accuracy: 0.8258
Precision: 0.768
Recall: 0.7568
F1 Score: 0.7624
```

```
summary = pd.DataFrame([
    {"Model": name, "Accuracy": accuracy_score(y_test, p), "Precision":
precision_score(y_test, p),
     "Recall": recall_score(y_test, p), "F1": f1_score(y_test, p)}
    for name, p in [("Random", random_predictions), ("Logistic Regression", lr_predictions),
("Linear SVM", svm_predictions)]
])
display(summary.round(4))

from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, svm_predictions)
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=["Not dup", "Dup"],
yticklabels=["Not dup", "Dup"])
plt.title("Linear SVM Confusion Matrix"); plt.xlabel("Predicted"); plt.ylabel("Actual");
plt.show()
```

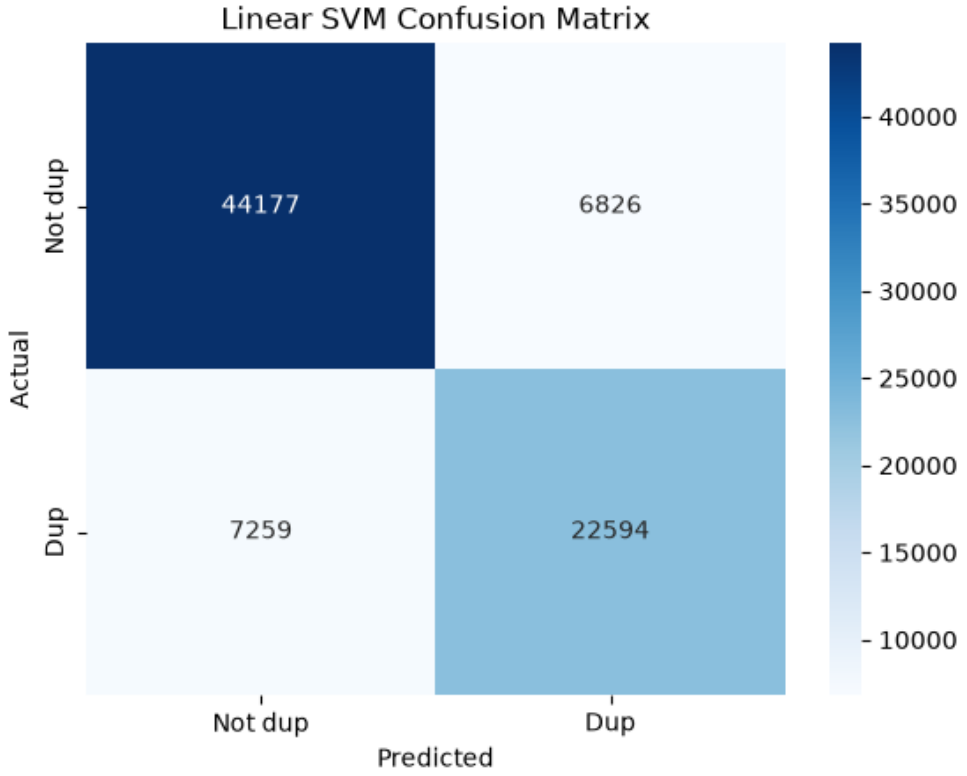
```
                Model  Accuracy  Precision  Recall      F1
0              Random    0.5008     0.3702  0.5021  0.4262
1  Logistic Regression    0.8168     0.7726  0.7140  0.7421
2           Linear SVM    0.8258     0.7680  0.7568  0.7624
```

VNRVJIET

WEEK: _____

NAME OF LAB: _____

EXPERIMENT NAME:




```
rows = []
for t in [0.3, 0.4, 0.5, 0.6, 0.7]:
    p = (lr_probabilities >= t).astype(int)
    rows.append({"threshold": t, "precision": precision_score(y_test, p), "recall":
recall_score(y_test, p), "f1": f1_score(y_test, p)})
display(pd.DataFrame(rows).round(4))
```

```
   threshold  precision  recall      f1
0        0.3     0.6519  0.8876  0.7517
1        0.4     0.7130  0.8102  0.7585
2        0.5     0.7726  0.7140  0.7421
3        0.6     0.8288  0.5966  0.6938
4        0.7     0.8871  0.4570  0.6033
```

VNRVJIET

NAME OF LAB: _____ WEEK: _____

EXPERIMENT NAME:

WEEK - 13

Title of the Experiment: Music generation using deep learning

Objective of the experiment:

- Understand how deep learning models can generate music.
- Explore Char-RNN and its role in music generation using ABC notation.
- Preprocess ABC music notation datasets for training.
- Build a Recurrent Neural Network (RNN) model for music sequence learning.
- Implement MIDI-based music generation using deep learning.
- Generate unique tabla music sequences using AI-based models

a. Real-world problem and music representation

Music generation using deep learning trains a model to learn patterns in existing music and generate new sequences. Applications include music composition, game soundtracks, musical assistance, and experimenting with different styles.

- ABC notation: represents music as readable text using characters and symbols for notes, rhythm, meter, and key.
- MIDI: represents musical events such as note pitch, note-on/note-off timing, velocity, and instrument information. It is not normally a recording of the actual sound.
- Objective: train a sequence model to predict the next character or musical event from previous ones.

b. Char-RNN with ABC notation, Char-RNN model

A Character-Level Recurrent Neural Network (Char-RNN) learns patterns in ABC notation one character at a time. An LSTM layer learns dependencies across characters, and the output layer predicts the next character.

For example, C D E G represents a sequence of notes. The model learns which characters and note patterns commonly follow one another.

```
abc = """X:1
T:Simple Rhythm
M:4/4
L:1/8
K:C
C D E G | G E D C |
C C G G | A A G2 |
```

```
"""

header = [l for l in abc.splitlines() if len(l) > 1 and l[1] == ":"]
body = [l for l in abc.splitlines() if not (len(l) > 1 and l[1] == ":")]
print("Header fields:", header)
print("Body lines   :", body)

bars = [b.strip() for line in body for b in line.split("|") if b.strip()]
print("\nBars found:", len(bars))
for i, b in enumerate(bars, 1):
    print(f"  bar {i}: {b!s:12} -> {len(b.split())} notes/units")
```

```
Header fields: ['X:1', 'T:Simple Rhythm', 'M:4/4', 'L:1/8', 'K:C']
Body lines   : ['C D E G | G E D C |', 'C C G G | A A G2 |']

Bars found: 4
  bar 1: C D E G      -> 4 notes/units
  bar 2: G E D C      -> 4 notes/units
  bar 3: C C G G      -> 4 notes/units
  bar 4: A A G2       -> 3 notes/units
```

```
text = "C D E G | G E D C |"
print("input char  -> target char")
for a, b in zip(text[:-1], text[1:]):
    print(f"   {a!r:5}   ->   {b!r}")
```

```
input char  -> target char
   'C'     ->   ' '
   ' '     ->   'D'
   'D'     ->   ' '
   ' '     ->   'E'
   'E'     ->   ' '
   ' '     ->   'G'
   'G'     ->   ' '
   ' '     ->   '|'
   '|'     ->   ' '
   ' '     ->   'G'
   'G'     ->   ' '
   ' '     ->   'E'
   'E'     ->   ' '
   ' '     ->   'D'
   'D'     ->   ' '
   ' '     ->   'C'
   'C'     ->   ' '
   ' '     ->   '|'
```

```
import numpy as np
rng = np.random.default_rng(0)

chars = ["C", "D", "E", "G", "|"]
probs = np.array([0.40, 0.20, 0.15, 0.15, 0.10])

print("argmax choice:", chars[int(np.argmax(probs))])
samples = rng.choice(chars, size=15, p=probs)
print("15 sampled choices:", " ".join(samples))
```

```
print("Observed frequencies:", {c: round(float(np.mean(samples == c)), 2) for c in chars})
```

```
argmax choice: C
15 sampled choices: E C C C G | E E D | G C G C E
Observed frequencies: {'C': 0.33, 'D': 0.07, 'E': 0.27, 'G': 0.2, '|': 0.13}
```

VNRVJIET

NAME OF LAB: _____ WEEK: _____

EXPERIMENT NAME:

WEEK - 14

c. Char-RNN with ABC notation: data preparation

```
import os, re, glob
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, TimeDistributed

np.random.seed(0); tf.random.set_seed(0)
os.makedirs("data/music", exist_ok=True)
```

```
music_data = """X:1
T:Simple Rhythm
M:4/4
L:1/8
K:C
C D E G | G E D C |
C C G G | A A G2 |
"""
with open("data/music/music.abc", "w", encoding="utf-8") as file:
    file.write(music_data)

print("ABC file created successfully!")
print(music_data)
```

```
ABC file created successfully!
X:1
T:Simple Rhythm
M:4/4
L:1/8
K:C
C D E G | G E D C |
C C G G | A A G2 |
```

```
cleaned_data = re.sub(r"(?m)^X:.*\n?", "", music_data).strip()

chars = sorted(set(cleaned_data))
char_to_num = {char: i for i, char in enumerate(chars)}
num_to_char = {i: char for char, i in char_to_num.items()}

sequences = np.array([char_to_num[c] for c in cleaned_data])
vocab_size = len(chars)

print("Cleaned ABC text:\n", cleaned_data)
print("Vocabulary:", chars)
print("Vocabulary size:", vocab_size)
print("Encoded sequence shape:", sequences.shape)
print("First 15 encoded:", sequences[:15])
```

```
Cleaned ABC text:
 T:Simple Rhythm
M:4/4
```

```
L:1/8
K:C
C D E G | G E D C |
C C G G | A A G2 |
Vocabulary: ['\n', ' ', '/', '1', '2', '4', '8', ':', 'A', 'C', 'D', 'E', 'G', 'K', 'L', 'M', 'R',
'S', 'T', 'e', 'h', 'i', 'l', 'm', 'p', 't', 'y', '|']
Vocabulary size: 28
Encoded sequence shape: (70,)
First 15 encoded: [18  7 17 21 23 24 22 19  1 16 20 26 25 20 23]
```

d. Char-RNN with ABC notation: many-to-many RNN, time distributed

```
SEQ_LEN = 40
X, y = [], []

for i in range(len(sequences) - SEQ_LEN):
    X.append(sequences[i:i + SEQ_LEN])
    y.append(sequences[i + 1:i + SEQ_LEN + 1])

X = np.array(X)
y = np.array(y)
print("X shape:", X.shape)
print("y shape:", y.shape)

print("\nWindow 0 input :", "".join(num_to_char[i] for i in X[0]).replace("\n", "/"))
print("Window 0 target:", "".join(num_to_char[i] for i in y[0]).replace("\n", "/"))
```

```
X shape: (30, 40)
y shape: (30, 40)

Window 0 input : T:Simple Rhythm/M:4/4/L:1/8/K:C/C D E G
Window 0 target: :Simple Rhythm/M:4/4/L:1/8/K:C/C D E G |
```

e. Char-RNN with ABC notation: model architecture

```
model = Sequential([
    Embedding(input_dim=vocab_size, output_dim=32),
    LSTM(64, return_sequences=True),
    TimeDistributed(Dense(vocab_size, activation="softmax")),
])
model.compile(optimizer="adam", loss="sparse_categorical_crossentropy")
model.build(input_shape=(None, SEQ_LEN))
model.summary()
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >=
2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-
DirectML plugin.
```

```
Model: "sequential"
```

```
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━┓
┃ Layer (type)                    ┃ Output Shape           ┃       Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━┩
│ embedding (Embedding)           │ (None, 40, 32)         │           896 │
├─────────────────────────────────┼────────────────────────┼───────────────┤
```

VNRVJIET

NAME OF LAB: _____ WEEK: _____

EXPERIMENT NAME:

```
| lstm (LSTM)                     | (None, 40, 64)          |        24,832 |
|---------------------------------|-------------------------|---------------|
| time_distributed                | (None, 40, 28)          |         1,820 |
| (TimeDistributed)               |                         |               |
```

```
Total params: 27,548 (107.61 KB)
```

```
Trainable params: 27,548 (107.61 KB)
```

```
Non-trainable params: 0 (0.00 B)
```

```
history = model.fit(X, y, epochs=30, batch_size=32, verbose=2)
```

```
Epoch 1/30
1/1 - 7s - 7s/step - loss: 3.3326
Epoch 2/30
1/1 - 0s - 118ms/step - loss: 3.3217
Epoch 3/30
1/1 - 0s - 109ms/step - loss: 3.3102
Epoch 4/30
1/1 - 0s - 123ms/step - loss: 3.2973
Epoch 5/30
1/1 - 0s - 128ms/step - loss: 3.2826
Epoch 6/30
1/1 - 0s - 108ms/step - loss: 3.2652
Epoch 7/30
1/1 - 0s - 104ms/step - loss: 3.2446
Epoch 8/30
1/1 - 0s - 109ms/step - loss: 3.2200
Epoch 9/30
1/1 - 0s - 140ms/step - loss: 3.1902
Epoch 10/30
1/1 - 0s - 113ms/step - loss: 3.1539
Epoch 11/30
1/1 - 0s - 114ms/step - loss: 3.1094
Epoch 12/30
1/1 - 0s - 116ms/step - loss: 3.0554
Epoch 13/30
1/1 - 0s - 186ms/step - loss: 2.9911
Epoch 14/30
1/1 - 0s - 139ms/step - loss: 2.9179
Epoch 15/30
1/1 - 0s - 109ms/step - loss: 2.8392
Epoch 16/30
1/1 - 0s - 117ms/step - loss: 2.7615
Epoch 17/30
1/1 - 0s - 107ms/step - loss: 2.6924
Epoch 18/30
1/1 - 0s - 106ms/step - loss: 2.6390
Epoch 19/30
1/1 - 0s - 112ms/step - loss: 2.6045
Epoch 20/30
1/1 - 0s - 120ms/step - loss: 2.5856
Epoch 21/30
1/1 - 0s - 106ms/step - loss: 2.5740
Epoch 22/30
```

```
1/1 - 0s - 112ms/step - loss: 2.5626
Epoch 23/30
1/1 - 0s - 103ms/step - loss: 2.5476
Epoch 24/30
1/1 - 0s - 108ms/step - loss: 2.5286
Epoch 25/30
1/1 - 0s - 103ms/step - loss: 2.5069
Epoch 26/30
1/1 - 0s - 127ms/step - loss: 2.4839
Epoch 27/30
1/1 - 0s - 107ms/step - loss: 2.4611
Epoch 28/30
1/1 - 0s - 167ms/step - loss: 2.4397
Epoch 29/30
1/1 - 0s - 154ms/step - loss: 2.4205
Epoch 30/30
1/1 - 0s - 146ms/step - loss: 2.4037
```

```
def generate_music(seed, length=200):
    output = seed
    for _ in range(length):
        context = output[-SEQ_LEN:]
        encoded = np.array([[char_to_num[c] for c in context]])
        probabilities = model.predict(encoded, verbose=0)[0, -1]

        probabilities = probabilities / probabilities.sum()          # make sure they sum to
exactly 1
        next_id = np.random.choice(vocab_size, p=probabilities)
        output += num_to_char[next_id]
    return output

seed_text = cleaned_data[-SEQ_LEN:]
generated_music = generate_music(seed_text, length=200)
print(generated_music)

with open("data/music/generated_music.abc", "w", encoding="utf-8") as file:
    file.write(generated_music)
print("\nSaved: data/music/generated_music.abc")
```

```
C
C D E G | G E D C |
C C G G | A A G2 |CECC D/i| GCC|
GGL|G:G D |C  G4C
DDD|E 4E ED    C/|   D A   D   G G |:|CG
       EC C C| E G  C
G
D G| CCC |2GE G TCSEGC|D D
 D D   CCCDD p 2hGE |E| M D GGC  E4EL|K
 G C
G  |E
 DC |DCCG    |GC   2

Saved: data/music/generated_music.abc
```

```
history2 = model.fit(X, y, epochs=400, batch_size=32, verbose=0)
```

```
print("Loss after 30 epochs :", round(history.history["loss"][-1], 3))
print("Loss after 430 epochs:", round(history2.history["loss"][-1], 3))

print("\n--- Generated after longer training ---")
print(generate_music(seed_text, length=200))
```

```
Loss after 30 epochs : 2.404
Loss after 430 epochs: 0.083

--- Generated after longer training ---
C
C D E G | G E D C |
C C G G | A A G2 |2 G C | A |
C A C G |G |2 D |G C A G2 |2 A A C G | |hyyhythm
M4:4/4
L:1/8
K:C
C D E G | G E D C |
C C G G G |
C A G G2 C G | A G A G |G2 |G G |
C C G2 D E h A2G |2 |2 C2 | G G |C G G2 |hyG2 A G2 |2 2
```

g. MIDI music generation

```
import pretty_midi
os.makedirs("data/midi", exist_ok=True)

if not glob.glob("data/midi/*.mid"):
    # bol -> MIDI pitch (a stand-in mapping: low = bayan/bass strokes, high = dayan/treble
strokes)
    bol_pitch = {"Dha": 59, "Dhin": 56, "Na": 52, "Tin": 61, "Ta": 57, "Ge": 40, "Ke": 42,
"Tirakita": 49}
    teental = ["Dha", "Dhin", "Dhin", "Dha", "Dha", "Dhin", "Dhin", "Dha",
               "Dha", "Tin", "Tin", "Ta", "Ta", "Dhin", "Dhin", "Dha"]
    rng_m = np.random.default_rng(7)
    for k in range(2):
        pm = pretty_midi.PrettyMIDI()
        drum = pretty_midi.Instrument(program=0)
        t = 0.0
        for cycle in range(250):
            for bol in teental:
                if rng_m.random() < 0.12:                        # occasional variation (fill /
ornament)
                    bol = rng_m.choice(list(bol_pitch))
                drum.notes.append(pretty_midi.Note(velocity=int(rng_m.integers(70, 110)),
pitch=bol_pitch[bol],
                                                   start=t, end=t + 0.2))
                t += 0.25
        pm.instruments.append(drum)
        pm.write(f"data/midi/synthetic_tabla_{k + 1}.mid")
    print("No MIDI files found -> wrote 2 synthetic tabla-style files.")
else:
    print("Using the MIDI files already in data/midi/")
print(sorted(os.listdir("data/midi")))
```

```
No MIDI files found -> wrote 2 synthetic tabla-style files.
['synthetic_tabla_1.mid', 'synthetic_tabla_2.mid']
```

```
midi_files = glob.glob("data/midi/*.mid")
print("MIDI files found:", len(midi_files))

all_pitches = []
for path in midi_files:
    try:
        midi_data = pretty_midi.PrettyMIDI(path)
        for instrument in midi_data.instruments:
            for note in instrument.notes:
                all_pitches.append((note.start, note.pitch))
    except Exception as e:
        print("Skipping unreadable file:", path)

all_pitches.sort(key=lambda item: item[0])          # order the notes by when they start
pitches = np.array([pitch for _, pitch in all_pitches], dtype=np.int32)

print("Total notes:", len(pitches))
print("First pitches:", pitches[:20])
print("Distinct pitches used:", sorted(set(pitches.tolist())))
```

```
MIDI files found: 2
Total notes: 8000
First pitches: [59 59 56 56 56 56 59 59 61 59 56 56 56 56 59 59 59 59 61 61]
Distinct pitches used: [40, 42, 49, 52, 56, 57, 59, 61]
```

```
SEQ_LEN = 32

if len(pitches) <= SEQ_LEN:
    raise ValueError("Upload more MIDI files containing enough notes.")

X, y = [], []
for i in range(len(pitches) - SEQ_LEN):
    X.append(pitches[i:i + SEQ_LEN])
    y.append(pitches[i + SEQ_LEN])

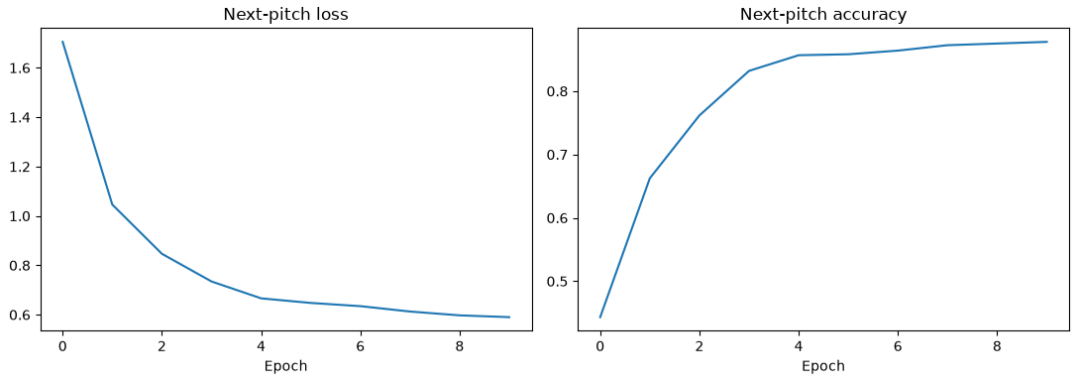
X = np.array(X)
y = np.array(y)
print("X:", X.shape, "| y:", y.shape)

model_midi = Sequential([
    Embedding(input_dim=128, output_dim=32),
    LSTM(64),
    Dense(128, activation="softmax"),
])
model_midi.compile(optimizer="adam", loss="sparse_categorical_crossentropy",
metrics=["accuracy"])
history_midi = model_midi.fit(X, y, epochs=10, batch_size=32, verbose=2)
```

```
X: (7968, 32) | y: (7968,)
Epoch 1/10
249/249 - 5s - 22ms/step - accuracy: 0.4433 - loss: 1.7050
Epoch 2/10
```

```
249/249 - 3s - 11ms/step - accuracy: 0.6624 - loss: 1.0461
Epoch 3/10
249/249 - 3s - 13ms/step - accuracy: 0.7617 - loss: 0.8465
Epoch 4/10
249/249 - 3s - 11ms/step - accuracy: 0.8320 - loss: 0.7345
Epoch 5/10
249/249 - 3s - 12ms/step - accuracy: 0.8567 - loss: 0.6661
Epoch 6/10
249/249 - 4s - 15ms/step - accuracy: 0.8582 - loss: 0.6476
Epoch 7/10
249/249 - 3s - 14ms/step - accuracy: 0.8640 - loss: 0.6346
Epoch 8/10
249/249 - 3s - 13ms/step - accuracy: 0.8725 - loss: 0.6127
Epoch 9/10
249/249 - 3s - 12ms/step - accuracy: 0.8751 - loss: 0.5973
Epoch 10/10
249/249 - 3s - 10ms/step - accuracy: 0.8778 - loss: 0.5900
```

```
import matplotlib.pyplot as plt
fig, axs = plt.subplots(1, 2, figsize=(11, 4))
axs[0].plot(history_midi.history["loss"]); axs[0].set_title("Next-pitch loss");
axs[0].set_xlabel("Epoch")
axs[1].plot(history_midi.history["accuracy"]); axs[1].set_title("Next-pitch accuracy");
axs[1].set_xlabel("Epoch")
plt.tight_layout(); plt.show()
```




```
seed = list(pitches[:SEQ_LEN])
generated_pitches = seed.copy()

for _ in range(100):
    context = np.array([generated_pitches[-SEQ_LEN:]])
    probabilities = model_midi.predict(context, verbose=0)[0]
    next_pitch = np.random.choice(128, p=probabilities / probabilities.sum())
    generated_pitches.append(int(next_pitch))

output = pretty_midi.PrettyMIDI()
piano = pretty_midi.Instrument(program=0)

for i, pitch in enumerate(generated_pitches[SEQ_LEN:]):
    start = i * 0.3
    end = start + 0.25
```

```
    piano.notes.append(pretty_midi.Note(velocity=90, pitch=int(pitch), start=start, end=end))

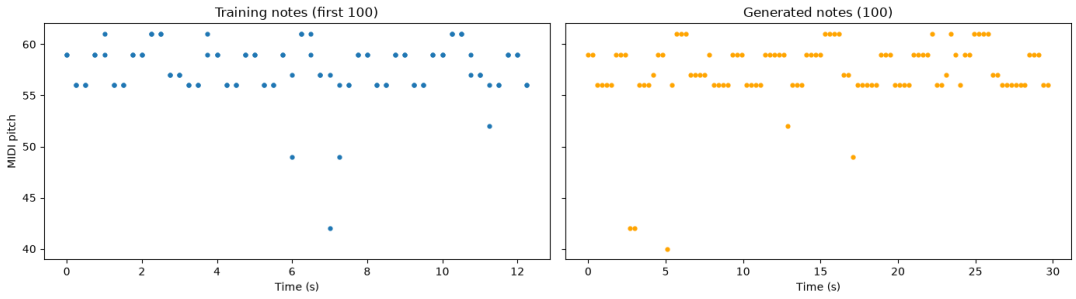
output.instruments.append(piano)
output.write("data/music/generated_midi.mid")
print("Generated MIDI saved successfully.")
print("Generated pitches:", generated_pitches[SEQ_LEN:SEQ_LEN + 20], "...")
```

```
Generated MIDI saved successfully.
Generated pitches: [59, 59, 56, 56, 56, 56, 59, 59, 59, 42, 42, 56, 56, 56, 57, 59, 59, 40, 56, 61]
...
```

```
gen = pretty_midi.PrettyMIDI("data/music/generated_midi.mid")
gen_notes = gen.instruments[0].notes

fig, axs = plt.subplots(1, 2, figsize=(14, 4), sharey=True)
orig = sorted(all_pitches)[:100]
axs[0].scatter([s for s, _ in orig], [p for _, p in orig], s=12); axs[0].set_title("Training
notes (first 100)")
axs[1].scatter([n.start for n in gen_notes], [n.pitch for n in gen_notes], s=12,
color="orange"); axs[1].set_title("Generated notes (100)")
for a in axs: a.set_xlabel("Time (s)")
axs[0].set_ylabel("MIDI pitch"); plt.tight_layout(); plt.show()

print("Generated file duration (s):", round(gen.get_end_time(), 1), "| notes:",
len(gen_notes))
print("Pitches used by generated music that were never in the training data:",
      sorted(set(n.pitch for n in gen_notes) - set(pitches.tolist())))
```




```
Generated file duration (s): 30.0 | notes: 100
Pitches used by generated music that were never in the training data: []
```